

分类号 TP311.1

学号 20212060756

密级



西安石油大学
Xi'an Shiyou University

全日制专业学位

硕士学位论文

题目 基于 uni-app 与 Spring Boot 框架的 Web

应用开发平台的设计与实现

论文作者 吴迁

指导教师 张仙伟 教授

专业、领域名称 计算机技术

提交论文时间 2025 年 5 月 30 日

论文题目：基于 uni-app 与 Spring Boot 框架的 Web 应用开发平台的设计与实现

专 业：计算机技术

硕 士 生：吴迁(签名) _____

导 师：张仙伟(签名) _____

摘 要

随着数字化转型加速和移动互联网的快速发展，中小型企业 Web 应用开发中普遍面临基础功能重复开发、跨平台适配成本高、开发周期紧张等关键问题。现有开发平台虽提供了部分解决方案，但仍存在技术适配性不足、功能冗余度高以及扩展能力有限等瓶颈。针对上述问题，本文提出了一种基于 uni-app 与 Spring Boot 框架的轻量级 Web 应用开发平台，通过模块化设计减少冗余功能开发，通过标准化接口提升跨端开发效率，从而降低开发门槛。

该平台采用前后端分离架构，后端采用单体架构实现轻量化部署，适配中小型团队的快速迭代需求。在技术选型上，前端基于 uni-app 框架进行开发，使用 Pinia 管理全局状态，基于条件编译机制聚焦 Web 端，在避免代码冗余问题的同时，也保留了向多端扩展的能力。后端基于 Spring Boot 框架，通过 Starter 快速集成 Redis 缓存、MySQL 数据库等中间件，基于 Sa-Token 权限框架实现 RBAC 模型，支持动态的路由拦截与细粒度的权限校验。

该平台主要包括四个功能模块：系统配置模块、权限管理模块、系统监控模块以及开发工具模块。系统配置模块通过统一管理系统参数和数据字典，实现业务逻辑的实时热更新；权限管理模块基于 RBAC 模型，支持目录-页面-按钮三级权限动态分配；系统监控模块提供运行状态的实时展示与日志管理功能；开发工具模块提供了面向 uni-app 组件的可视化表单构建和基于模板的代码生成功能。

系统测试与开发实践表明，该平台在功能需求与非功能需求两方面达到了预期目标，验证了方案的工程可行性与技术有效性。本文为中小型企业提供了一套高效、可扩展的 Web 应用开发解决方案，具有实际应用价值与推广潜力。

关键词：Web 应用 快速开发 权限管理 uni-app Spring Boot

论文类型：应用研究

Subject : Design and Implementation of a Web Application Development Platform Based on uni-app and Spring Boot Framework

Speciality: Computer Technology

Name : Wu Qian(signature)_____

Instructor: Zhang Xianwei(signature)_____

ABSTRACT

With the acceleration of digital transformation and the rapid development of mobile internet, small and medium-sized enterprises commonly face critical challenges in web application development, including redundant development of basic functionalities, high cross-platform adaptation costs, and tight development cycles. While existing development platforms offer partial solutions, issues such as insufficient technical adaptability, high functional redundancy, and limited scalability persist. To address these problems, this paper proposes a lightweight web application development platform based on the uni-app and Spring Boot frameworks. It employs modular design to reduce redundant functional development and utilizes standardized interfaces to enhance cross-platform development efficiency, thereby lowering the technical barrier for developers.

The platform adopts a frontend-backend separation architecture, with a monolithic backend design to enable lightweight deployment, catering to the rapid iteration needs of SMEs. Technically, the frontend is developed using the uni-app framework, manages global states with Pinia, and focuses on the web terminal through a conditional compilation mechanism to avoid code redundancy while retaining multi-platform extensibility. The backend, built on Spring Boot, integrates Redis caching, MySQL databases, and other middleware via Starter components. Combined with the Sa-Token permission framework, it implements a Role-Based Access Control model supporting dynamic route interception and fine-grained permission verification.

The platform primarily consists of four functional modules. The system configuration module unifies the management of system parameters and data dictionaries to achieve real-time hot updates of business logic. The permission management module, based on the RBAC model, supports dynamic allocation of three-tier permissions at the directory, page, and button levels. The system monitoring module provides real-time visualization of operational status and log management capabilities. The development toolkit module offers visual form construction for uni-app components and template-based code generation functionalities.

System testing and development practices demonstrate that the platform meets both functional and non-functional requirements, validating its engineering feasibility and technical

effectiveness. This study provides small and medium-sized enterprises with an efficient and scalable web application development solution, demonstrating practical value and potential for broader adoption.

Keywords: Web application; Rapid development; Permission management; uni-app; Spring Boot

Thesis: Application Research

目 录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.3 主要研究内容.....	3
1.4 章节安排.....	4
第二章 相关技术介绍	5
2.1 uni-app	5
2.2 Pinia	6
2.3 Spring Boot	6
2.4 Sa-Token	7
2.5 Free Marker.....	7
2.6 MyBatis-Plus.....	8
2.7 本章小结.....	8
第三章 系统需求分析	9
3.1 系统需求概述.....	9
3.2 系统功能性需求分析.....	9
3.3 系统非功能性需求分析.....	12
3.4 本章小结.....	13
第四章 系统的总体设计	14
4.1 系统总体架构设计.....	14
4.2 系统功能模块划分.....	15
4.3 数据库设计.....	15
4.3.1 概念模型设计.....	15
4.3.2 数据库表设计.....	17
4.4 本章小结.....	22
第五章 系统的详细设计与实现	23
5.1 系统开发环境与项目搭建	23
5.1.1 系统开发环境.....	23
5.1.2 项目的搭建.....	23
5.2 系统配置模块.....	24
5.2.1 字典管理.....	25
5.2.2 参数管理.....	26
5.3 权限管理模块.....	27
5.3.1 用户管理.....	27

5.3.2 角色管理.....	31
5.3.3 菜单管理.....	35
5.4 系统监控模块.....	37
5.4.1 数据监控.....	37
5.4.2 性能监控.....	38
5.4.3 系统日志.....	39
5.5 开发工具模块.....	41
5.5.1 表单构建.....	41
5.5.2 代码生成.....	45
5.6 本章小结.....	49
第六章 系统测试	50
6.1 测试环境.....	50
6.2 系统功能测试.....	50
6.3 系统非功能测试.....	56
6.4 本章小结.....	57
第七章 总结与展望	58
参考文献.....	59

第一章 绪论

1.1 研究背景与意义

Web 应用开发平台（又称 Web 开发框架）是一类用于 Web 软件产品研发的辅助工具。此类平台是一种可扩展的中间件系统。一方面，Web 应用开发平台提供了一些系统通用的或者面向特定领域相关业务的功能模块，开发人员可以直接复用这些功能，并在此基础上进行二次开发来实现最终能满足实际需求的目标系统；另一方面，Web 应用开发平台还提供了代码生成、系统监控、可视化配置等功能，用于辅助软件产品的研发^[1]。Web 应用开发平台能有效提高研发效率和产品质量，广泛应用于国内外的 Web 开发领域^[2]。

随着移动互联网的快速发展与智能设备的广泛普及，用户对跨平台软件服务的需求显著增长^[3]。软件产品须提供多终端场景下的一致化服务，Web 应用通常须额外适配各种移动端和小程序端，针对每个终端分别编写独立代码，导致开发与维护成本显著增加。近年来，市面上出现了一些跨平台开发框架，此类框架能通过代码转换或条件编译等方式将一套代码适配到多个终端，提高了代码复用率，在一定程度上实现了一套前端代码，多端打包发布的能力。

目前，中小型企业 Web 应用开发中普遍面临基础功能重复开发、跨平台适配成本高、开发周期紧张等关键问题。现有开发平台虽提供了部分解决方案，但仍存在技术陈旧、功能冗余以及扩展能力有限等问题。针对上述问题，本文设计并实现了一个基于 uni-app 与 Spring Boot 框架的轻量级 Web 应用开发平台，通过模块化设计与标准化接口降低开发门槛并提升开发效率。

平台通过以下设计弥补现有技术方案的不足：

（1）基于 uni-app 的条件编译机制，将代码适配聚焦于 Web 端，在避免冗余代码的同时保留了向多端扩展的能力。开发人员可以基于 uni-app 的跨平台组件和通用 API 实现高效的代码复用，通过响应式布局解决 Web 项目在不同设备或分辨率下显示和操作的兼容性问题；

（2）基于分层架构与开发规范，聚焦前后端开发工具的规范化，封装通用工具类和可复用的 API 接口，提升复用率与协作效率，降低联调成本；

（3）将系统通用功能封装为独立的模块，实现对系统配置、权限管理等基础功能的复用，减少重复编码，同时提供 uni-app 表单构建功能以及基于模板的代码生成功能，实现系统开发效率的提升。

1.2 国内外研究现状

近年来，随着 Web 应用开发技术的快速发展，国内外学者与企业围绕跨平台开发技术、Web 应用开发平台等领域展开了广泛研究。

(1) 跨平台开发技术研究现状

在跨平台开发领域，国内外研究主要聚焦于多端适配与代码复用效率的提升^[4]。国外早期框架如 React Native 与 Flutter 通过虚拟 DOM 与自绘引擎技术，实现了移动端的高性能渲染，但对 Web 原生特性的支持存在局限性^[5,6]。例如，React Native 的 Web 兼容性需依赖第三方库，导致开发复杂度增加^[7]；Flutter 的 Web 版本在动态布局与浏览器兼容性上仍存在明显的性能损耗^[8]。相比之下，国内技术生态更注重轻量化与场景适配，uni-app 凭借其创新的条件编译机制，使开发者能够基于统一代码库实现跨平台开发，在确保 Web 原生体验的同时，深度适配 iOS/Android 原生 APP 以及主流小程序平台，并通过分平台编译策略实现各端性能优化与特性扩展^[9]。Taro 是一款基于 React/Vue 双技术栈的跨端框架，主要支持小程序生态并支持扩展至 H5 与 React Native，其维护更新频率和社区生态弱于上述三类框架。为清晰呈现各框架的技术特点与能力差异，下表将从核心技术、平台覆盖范围、Web 支持特性以及生态成熟度等关键维度进行对比，具体细节如表 1-1 所示。

表 1-1 跨平台开发技术基础差异对比

维度	uni-app	Taro	Flutter	React Native
底层技术	Vue.js	Vue.js/React	Dart	React
主要优势	多端覆盖广 条件编译灵活	小程序生态适配 性强	自绘引擎 UI 高度定制	原生组件桥接 社区生态成熟
跨平台支持	H5/小程序 Andriod/iOS	H5/小程序 React Native	Web/桌面端 Android/iOS	Android/iOS
性能	接近原生	中等	接近原生	中等
学习曲线	低	中等	高	中等
生态支持	插件市场丰富	扩展性受限	插件较少	第三方库丰富
包体积	较小	中等	较大	较小
维护状态	更新稳定	更新较慢	更新稳定	更新稳定

(2) Web 应用开发平台发展现状

在 Web 应用开发平台的研究中，国内外学者普遍关注平台模块化与自动化工具的集成。国外开源项目如 JHipster 通过代码生成器实现基础 CRUD 功能的自动化构建，但其权限管理模块依赖 Spring Security，配置复杂且灵活性较低；国内开源项目 Jeecg-Boot 虽提供了可视化表单设计器，但其功能过于冗余，无法满足中小企业快速迭代的需求^[10]。从行业应用来看，大型企业倾向于采用微服务架构与低代码平台构建复杂系统，但其高昂的运维成本与技术要求超出了中小企业的承受能力。国内部分中小企业尝试通过开源框架搭建内部开发平台，却因模块化程度不足导致频繁的功能重复开发。

目前，国内外也陆续出现了一些低代码开发平台^[11]。低代码开发一般是指通过拖放

和可视化配置等方式来创建应用程序，甚至无须编写任何代码。这种低代码开发平台的用户可以是业务人员，用户无须拥有专业的开发知识，只须对业务足够了解，便可使用平台实现目标系统，满足业务的功能需求^[12]。低代码开发平台能够使用户专注于业务逻辑，提高开发效率^[15]。国内主流的低代码开发平台包括宜搭、织信、简道云等平台^[13]。国外主流的低代码开发平台包括 Outsystems、Mendix、Celonis 等平台^[14]。虽然这种开发模式门槛较低，但是却无法较好地满足复杂的业务需求以及应用程序的高度定制化要求，这类平台须提供足够数量和类别的模板，才可应对各种业务需求。低代码开发平台灵活性较低，功能完全依赖于低代码平台，平台的使用者对业务逻辑的实现完全不了解，故障或问题的解决以及系统的运维将全部依赖于低代码开发厂商^[15-17]。

2020 年，吴昌政^[18]针对传统一体化框架的扩展性瓶颈，提出基于前后端分离的 Web 开发框架，通过微服务架构、云原生部署及灰度发布机制，实现系统模块化解耦与能力开放化，支撑运营商 IT 系统向“平台+应用”模式转型，使新应用上线周期缩短 40%。

2021 年，刘启伟^[19]针对前端开发效率低及 UI 协同难题，基于 Vue.js 设计拖拽式可视化工具，通过分层架构实现模块化页面设计与代码自动生成，测试表明该工具使重复编码量减少 45%，开发周期缩短 35%，有效缓解多角色协作成本。

2022 年，郭文学^[20]针对中小团队开发效率瓶颈，基于 Vue+SpringBoot 构建低代码工具，通过 YAML 配置自动生成表单/统计界面及后端逻辑，封装权限管理等通用模块并支持二次开发，使基础功能开发效率提升 60%，成功应用于跨境超市系统及高校小程序。

2023 年，康雯珺^[21]针对低代码平台组件碎片化与交互逻辑复杂问题，提出基于 SPA 的渲染引擎及统一组件体系，实现可视化单页应用动态编排，测试表明组件复用率提升 50%，多维度编排效率提高 40%

2024 年，王雨霏^[22]面向多租户三维可视化需求，设计通用组件描述模型与多引擎架构，通过四层 SaaS 服务实现数据隔离与交互配置，使大屏开发周期缩短 60%，多源数据整合效率提升 45%。

1.3 主要研究内容

本文围绕中小型企业 Web 应用开发中普遍面临的基础功能重复开发、跨平台适配成本高、开发周期紧张等关键问题，提出并实现了一个基于 uni-app 与 Spring Boot 框架的轻量级 Web 应用开发平台。主要研究内容如下：

（1）开发技术的选型。调研跨平台开发技术的研究现状和 Web 应用开发平台的发展现状。分析中小型企业 Web 应用开发中普遍面临的一些问题，确定系统前后端开发所采用的技术。系统主要基于 uni-app 与 Spring Boot 框架。

（2）系统需求分析。对系统需求进行深入调研，根据 Web 开发现状和实际应用场

景，分析系统的功能性需求和非功能性需求。

(3) 对系统进行总体设计。设计系统的总体结构和划分系统的功能模块。确定系统应该提供的所有功能或特性。根据需求分析结果，设计数据库表结构。

(4) 对系统进行详细设计与实现。配置系统开发环境和搭建前后端项目。对系统的各个功能模块进行详细设计与实现，实现基于 uni-app 与 Spring Boot 框架的 Web 应用开发平台，最后完成对系统的功能测试和非功能测试。

1.4 章节安排

本文围绕基于 uni-app 与 Spring Boot 框架的 Web 应用开发平台的设计与实现来展开。全文共分为七章，具体内容安排如下：

第一章是绪论。本章首先介绍了论文题目的研究背景与意义，然后阐述了国内外跨平台开发技术研究现状和 Web 应用开发平台发展现状。最后介绍了论文的主要研究内容和章节安排。

第二章是相关技术介绍。介绍了论文涉及的主要开发技术。包括前端框架 uni-app、状态管理库 Pinia、后端框架 Spring Boot、权限框架 Sa-Token、模板引擎 FreeMarker、数据库操作框架 MyBatis-Plus。

第三章是系统的需求分析。经过对需求的调研与分析，结合 Web 开发现状和实际应用场景，分析系统的功能性需求和非功能性需求。

第四章是系统的总体设计。对系统进行总体架构设计和功能模块划分，然后根据需求分析结果进行数据库设计。

第五章是系统的详细设计与实现。对系统的各个功能模块进行了详细设计与实现。

第六章是系统测试。对 Web 应用开发平台进行功能测试和非功能测试。

第七章是总结与展望。总结系统设计与实现过程中的主要工作内容，分析了目前存在的一些问题，指出系统需要完善和改进的方面。

第二章 相关技术介绍

2.1 uni-app

uni-app 是一款基于 Vue.js 的跨平台应用开发框架^[23]，允许开发者通过单一代码库同步生成 Web 端、Android、iOS 及多种小程序应用，在一定程度上能实现一套前端代码，多端打包发布的能力。其核心优势在于通过条件编译实现平台差异化适配，同时整合了原生渲染与 WebView 混合技术，在开发效率与性能体验之间取得平衡。uni-app 显著降低了多端开发的学习门槛^[24]。uni-app 通过扫描开发人员在所有可遍历的项目文件中编写的编译指令，根据匹配的目标平台标识符，移除不满足条件或未成功匹配的代码，最后将处理后的代码按照目标平台的规则进行打包，以此来实现条件编译。uni-app 功能框架图如图 2-1 所示^[25]。



图 2-1 uni-app 功能框架图

uni-app 具有如下特点：

- (1) 入门门槛低：基于 Vue.js 技术栈和小程序 api，语法简洁，没有额外的学习成本；
- (2) 全端兼容：在跨平台的同时，通过差异化编译策略，可以直接调用各平台特有原生接口，编写个性化代码，实现该平台的专有功能；
- (3) 体验趋近原生：采用动态渲染引擎，能自动 diff 更新数据和实现原生渲染，自动匹配最优性能方案；

(4) 开发效率高: uni-app 生态丰富, 开发案例众多, 基于 HBuilderX 能实现多端同步调试与快速交付。

2.2 Pinia

Pinia 是一款用于管理 Web 应用前端状态的数据存储库^[26], 是 Vue.js 生态中新一代状态管理解决方案, 专注于提升前端复杂场景下的数据流管理效率。其设计目标在于解决传统状态管理库在复杂性与开发体验上的不足。Pinia 主要特点如下:

(1) 移除了传统模式中的 mutations 层, 允许开发者通过 actions 直接操作状态, 结合组合式 API 按功能聚合业务逻辑, 降低了模块之间的耦合度;

(2) 内置了完善的类型声明, 原生支持 TypeScript, 提供了开箱即用的类型推导能力, 降低类型错误风险, 增强代码健壮性;

(3) 支持多 Store 灵活组织业务模块, 规避了单一数据源导致的代码膨胀问题, 为应用的状态管理提供了模块化解决方案;

(4) 基于 Vue 3 的响应式能力, 能自动追踪状态依赖, 减少了冗余渲染和性能开销;

(5) 内置了调试工具链, 兼容 Vue 2 项目, 简化异步逻辑与跨组件通信。

相比传统方案 Vuex, Pinia 凭借简洁直观的 API 设计, 既适用于中小型应用的快速搭建, 又能支撑大型项目的长期迭代, 兼顾了不同规模项目的需求, 成为平衡开发效率与维护复杂度的更优选择。

2.3 Spring Boot

Spring Boot 是应用最广泛的轻量化 Java 开发框架, 其核心理念是通过“开箱即用”的设计简化 Spring 生态的复杂配置^[27]。它整合了内嵌的 Web 容器、自动化依赖管理和生产监控等能力^[28], 使开发者能快速构建独立运行的企业级应用^[29], 兼容单体架构和微服务场景, 能大幅提升开发与部署效率。Spring Boot 主要特性如下:

(1) 基于约定与默认值自动完成基础设置, 仅需极简代码即可运行完整应用, 提升了项目的开发效率;

(2) 提供 starter 系列依赖包, 按需引入标准化依赖, 自动关联版本兼容的第三方库, 避免依赖包的版本冲突问题;

(3) 根据类路径与配置智能加载组件, 开发者可通过配置文件改变默认行为;

(4) 集成了健康检查、性能指标等运维端点, 支持与 Prometheus 等监控系统对接, 提供了生产级运维能力;

(5) 支持 Profile 隔离开发、测试、生产配置, 实现多环境无缝切换, 提升了部署的灵活性。

Spring Boot 基于上述特性, 在降低开发复杂度的同时, 也平衡了开发效率与架构灵

活性，成为目前 Java 开发领域的关键技术选型。其成熟的社区生态与持续迭代，进一步巩固了在 Java 全栈开发中的核心地位^[30]。

2.4 Sa-Token

Sa-Token 是一款轻量级 Java 权限认证框架^[31]，主要面向登录认证、权限校验以及会话管理等关键场景提供标准化解决方案。该框架通过注解驱动与声明式配置降低开发复杂度，支持细粒度权限控制、令牌自动续期等功能特性，并可满足多终端并发登录、用户强制下线等业务需求。在架构层面，Sa-Token 不强制耦合外部组件，具备低侵入性与开箱即用的特性，适用于单体应用或微服务体系，能够帮助开发者通过较少的代码快速构建高安全性的权限管理系统，尤其适合需快速迭代的中小型项目。

相较于传统安全框架，Sa-Token 在轻量化设计与学习成本控制方面表现突出，其原生支持微服务架构并集成分布式会话管理机制；Apache Shiro 作为基础安全框架虽功能完备，但需依赖额外插件实现分布式会话等扩展能力，其灵活性较高但存在一定的学习门槛，更适用于需自定义安全逻辑的单体系统；Spring Security 凭借与 Spring 生态的深度整合，在复杂权限策略支持上具有优势，但其配置复杂度较高且依赖过滤器链机制，通常面向需深度定制安全策略的企业级应用场景^[32]。综合而言，Sa-Token 以功能精简与开发效率为核心竞争力，Shiro 侧重于基础安全能力的可扩展性^[33]，而 Spring Security 的优势则集中于生态兼容性与对高级安全需求的支持。

2.5 FreeMarker

FreeMarker 是一款基于 Java 的模板引擎，专注于将动态数据与静态模板结合生成文本输出。开发者通过编写静态模板文件定义展示结构，由数据模型填充内容，从而实现视图层与业务逻辑的解耦。FreeMarker 无需复杂的运行时环境，仅依赖数据模型即可生成内容^[34]。FreeMarker 主要应用于 Web 页面生成、基于模板的代码生成等场景。相较于 Velocity 和 Thymeleaf，FreeMarker 不仅提供了更丰富的指令集与类型安全机制；还凭借其强类型模板引擎与编译器优化策略，在复杂逻辑处理与性能上更具优势。FreeMarker 具有如下特点：

(1) 模板只通过占位符和指令处理数据，不嵌入 Java 代码，通过纯声明式的模板驱动将视图与数据分离；

(2) 支持嵌套对象的自动解析与空值处理，默认开启特殊字符转义防御 XSS 攻击；

(3) 模板加载时会执行静态语法检查，运行时强制空值敏感策略，提供了从解析到生成全生命周期的健壮性保障；

(4) 通过<#macro>参数化片段、<#include>子模版以及 TemplateDirectiveModel 自定义指令实现了组件的复用和扩展。

2.6 MyBatis-Plus

MyBatis-Plus 是在 MyBatis 框架的基础之上，对数据库的表操作、SQL 语句构建等一系列功能进行了增强和封装的一个扩展工具包^[35]。MyBatis-Plus 通过一系列通用的数据访问接口和 API 驱动式的条件组合构造器等策略，显著降低了基于 MyBatis 进行基础功能开发的工作量和复杂度，使开发人员能更加聚焦于业务逻辑的功能实现。

MyBatis-Plus 具有如下特点：

- (1) 通过 `BaseMapper<T>` 接口预置了类型化的基础操作方法，实现零 SQL 代码的通用数据操作；
- (2) 基于 Lambda 或 Wrapper 的链式 API 动态构建类型安全的查询条件，杜绝 SQL 注入风险；
- (3) 通过拦截器链无侵入式扩展了分页、乐观锁等原生 MyBatis 未内置的功能；
- (4) 实体类通过继承 `Model<T>` 直接获得持久化能力，简化领域模型与存储层的交互逻辑。

2.7 本章小结

本章系统地介绍了论文涉及到的主要技术。首先介绍了前端开发框架 uni-app 和状态管理工具 Pinia，然后介绍了后端开发框架 Spring Boot 和权限管理框架 Sa-Token，最后介绍了模板引擎 FreeMarker。本章为后续章节的需求分析、系统设计与实现奠定了技术基础。

第三章 系统需求分析

本章基于对中小型企业 Web 应用开发中面临的核心问题进行调研与分析,结合系统的目标定位,对系统的功能性需求和非功能性需求进行梳理。通过用例图进行功能模块分解,明确系统的核心功能边界与交互逻辑,为后续设计与实现奠定基础。

3.1 系统需求概述

当前中小型企业在 Web 应用开发中普遍面临基础功能重复开发、多端适配成本高、开发周期紧张等关键问题。传统开发模式下,团队需针对不同平台独立维护多套前端代码,导致技术栈分散、代码复用率低下;新项目的权限管理、日志监控等通用模块往往须从零开始构建,导致开发效率降低;现有解决方案常因过度追求架构复杂度而增加运维负担,难以适配资源有限的中小型项目需求。

针对上述问题,本文提出一种基于 uni-app 与 Spring Boot 框架的轻量级 Web 应用开发解决方案,聚焦于中小型企业的需求:

(1) 降低多端适配成本:前端采用 uni-app 框架,通过条件编译策略实现 H5 浏览器端的高效适配,避免多平台代码冗余问题,聚焦 Web 端场景,保证不同设备之间跨浏览器兼容性;

(2) 提升开发效率:后端基于 Spring Boot 构建模块化服务,提供基础的通用功能,减少重复开发工作量;通过统一管理系统参数和数据字典,实现业务逻辑的实时热更新;基于 RBAC 模型^[34],实现细粒度的动态权限管理;支持 uni-app 前端组件的可视化表单构建和前后端代码的模板化生成;

(3) 简化运维复杂度:采用单体架构设计,避免微服务带来的部署与维护压力^[36],适配资源有限的中小型企业;提供运行状态实时监控与日志管理功能。

3.2 系统功能性需求分析

根据实际需求,系统划分为四个功能模块:系统配置模块、权限管理模块、系统监控模块、开发工具模块。

(1) 系统配置模块

系统配置模块的主要功能包括参数管理和字典管理,参数管理主要是对系统的预置参数和可配置的业务参数进行动态管理,字典管理主要是对各种枚举数据的类型和内容进行集中管理。系统配置模块的用例图如图 3-1 所示。

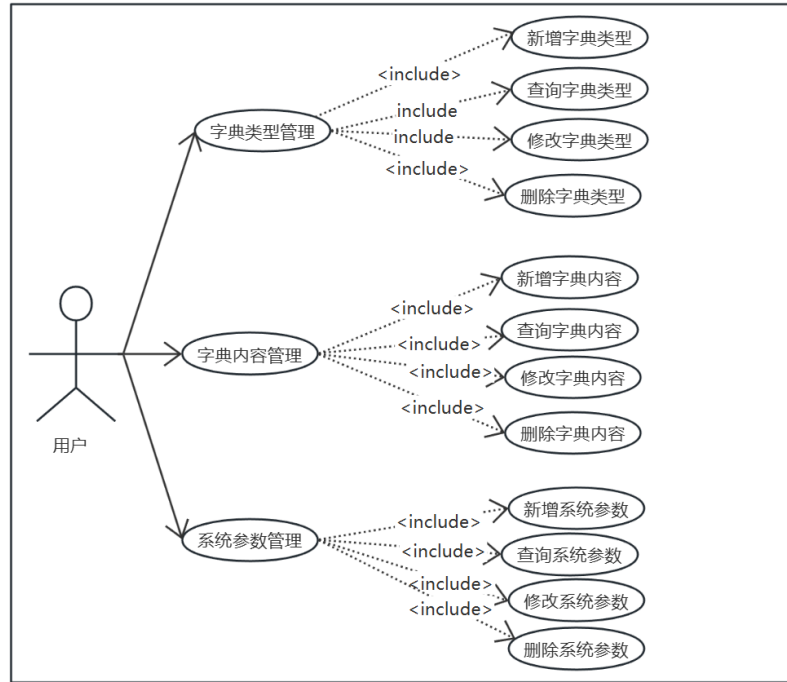


图 3-1 系统配置用例图

(2) 权限管理模块

权限管理模块主要包括用户管理、角色管理、菜单管理。系统应支持细粒度的权限控制，主要操作包括为用户分配角色、为角色分配用户、为角色分配菜单以及维护菜单之间的层级关系。权限管理模块的用例图如图 3-2 所示。

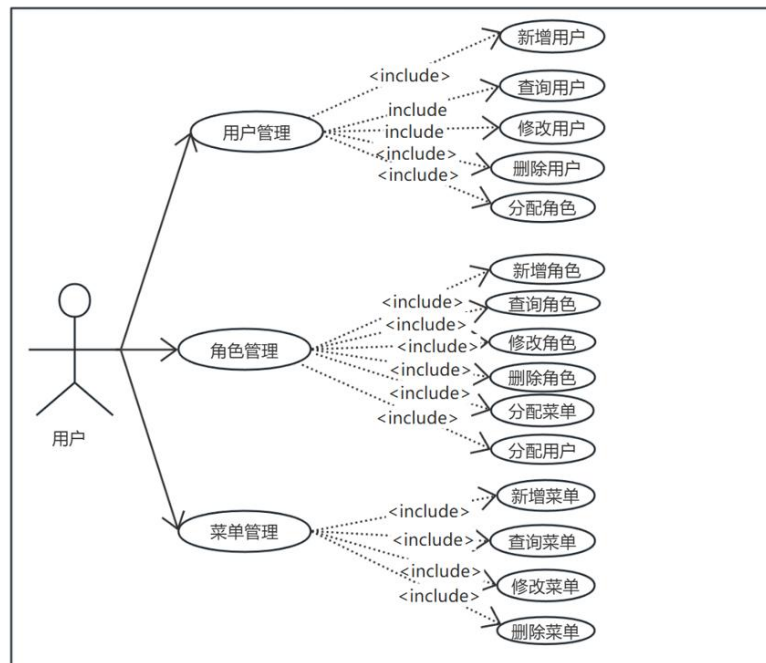


图 3-2 权限管理用例图

(3) 系统监控模块

系统监控模块主要功能包括数据监控、性能监控和系统日志。一方面，系统应实时

采集服务器 CPU、内存、磁盘等硬件资源占用率数据，并动态生成可视化图表进行展示。另一方面，系统应整合登录日志与操作日志查询功能，支持按时间、用户、操作类型等多维度筛选，辅助快速定位系统异常。数据监控模块的主要功能包括查看连接池监控、保存日志记录、查看 Redis 缓存列表、清除 Redis 缓存。性能监控模块主要包括查看系统环境参数、查看运行仪表盘。系统日志模块主要包括查看登录日志和查看操作日志。系统监控模块的用例图如图 3-3 所示。

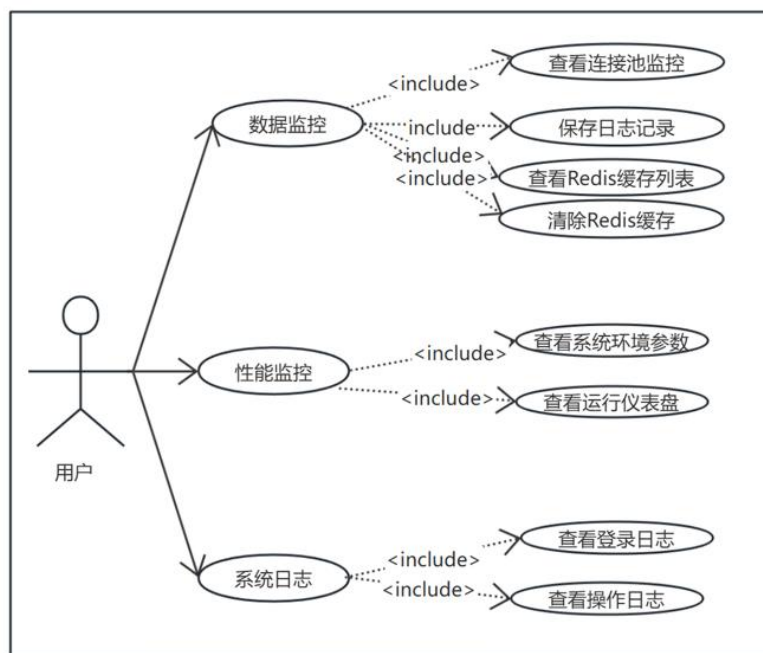


图 3-3 系统监控用例图

(4) 开发工具模块

开发工具模块主要包括可视化表单构建功能和基于模板的代码生成功能。表单构建功能主要包括拖拽组件到画布、配置组件或表单的参数、导出代码到文件、复制代码到剪切板。代码生成功能主要包括选择数据库表、配置生成参数、生成代码文件。开发工具模块的用例图如图 3-4 所示。

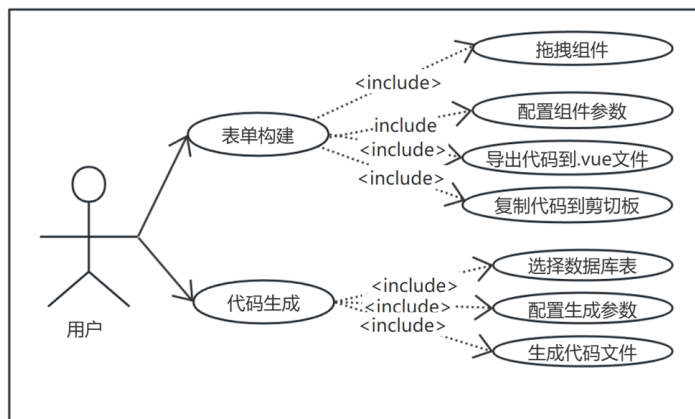


图 3-4 开发工具用例图

3.3 系统非功能性需求分析

非功能性需求是衡量软件系统整体质量的重要指标^[37]。本系统的主要非功能性需求如下：

（1）安全性

系统安全性是保障业务连续性与数据完整性的核心要素，直接关系到用户信任度与系统合规性。对于本系统而言，安全性设计需平衡易用性与风险控制，在提升开发效率的同时防范潜在的风险。具体需求包括：对常见安全风险进行前端过滤和后端拦截、对用户密码进行加密存储、基于 Sa-Token 框架实现细粒度的权限控制、关键的数据传输通过 HTTPS 加密、记录所有用户的登录日志和操作日志、敏感操作进行二次弹窗确认。

（2）兼容性

兼容性是衡量软件系统在多维度运行环境中保持功能一致性与稳定性的关键指标，具体涵盖对异构终端设备、多版本技术栈及动态业务场景的适配能力。对于本系统而言，兼容性设计需在开发效率与适配成本间取得平衡。具体需求包括：系统应通过响应式布局和媒体查询等技术保证前端跨平台 UI 组件在不同设备、不同分辨率下的显示一致性，并通过 CSS 前缀、渐进增强等策略确保主流浏览器的兼容性。系统应使用 JSON 格式数据与后端 RESTful 接口实现数据交互标准化。

（3）性能

性能是衡量软件系统能否高效支撑系统功能服务的核心指标，直接影响用户体验与系统扩展能力。对于本系统而言，性能优化需在资源有限条件下实现开发效率与运行稳定性的平衡，避免因性能瓶颈影响开发人员的开发过程和最终用户的使用体验。

（4）可维护性

可维护性是衡量软件系统长期可持续迭代能力的关键指标，直接影响开发团队应对需求变更、功能追加、缺陷修复以及架构优化的效率。为实现系统的可维护性，需通过规范化设计、模块化架构与组件化封装降低技术复杂性风险。规范化设计要求前后端工程遵循一致的目录结构、代码风格、异常处理机制及 API 响应格式，以降低团队协作成本；模块化架构通过功能解耦与分层设计减少模块间依赖关系，避免单一功能修改引发全局性影响；组件化封装则聚焦于通用 UI 组件、常用工具类等高频功能的抽象化与标准化接口设计，从而减少重复开发。此类策略通过提升代码可读性与可扩展性，为系统在快速迭代场景下的长期演进提供技术支撑。

（5）可扩展性

可扩展性是衡量系统适应业务增长与技术迭代能力的关键指标，决定了平台能否在需求变更或用户规模扩大时，通过模块化扩展而非重构满足新场景需求。对于本系统而言，可扩展性设计需在轻量化架构与长期技术演进需求间取得平衡，通过低耦合模块化设计与标准化接口规范，为功能扩展、多端适配及复杂业务场景提供灵活的技术实现。

3.4 本章小结

本章首先进行了系统需求概述，根据调研结果，明确了中小型企业的需求，然后通过用例图描述了系统的功能性需求，将系统划分为系统配置模块、权限管理模块、系统监控模块、开发工具模块，然后分析了系统主要的非功能需求，包括安全性、兼容性、性能、可维护性、可扩展性。后续章节将依据本章的需求分析结果展开系统的设计与实现。

第四章 系统的总体设计

4.1 系统总体架构设计

本系统采用前后端分离架构^[38],通过技术整合与模块化拆分实现轻量级 Web 应用开发开发平台。前端基于 uni-app 框架,主要负责展示界面与处理交互逻辑;后端基于 Spring Boot 框架,主要负责处理业务逻辑和持久化存储数据。前后端采用 HTTP/HTTPS 协议进行通信,通过 RESTful API 接口进行数据交互。系统的整体架构分为四层:

(1) 客户端层:用户通过客户端浏览器访问系统,系统通过响应式布局在客户端实现页面自适应,保证用户跨终端的操作体验;

(2) 前端层:负责用户界面渲染、交互逻辑处理以及 API 请求转发。基于 uni-app 内置组件和 API 进行前端开发,通过 Pinia 管理全局状态,封装通用的组件库用于代码复用和表单构建;

(3) 后端层:采用 Spring Boot 构建 RESTful API 服务,通过分层架构解耦业务逻辑与数据操作,集成 Sa-Token 实现细粒度的接口级权限管理,采用 MyBatis-Plus 简化数据访问操作;

(4) 数据存储层:提供数据持久化与缓存服务,保障数据一致性和访问效率。基于 MySQL 实现数据存储,基于 Redis 实现数据缓存。

系统的总体架构如图 4-1 所示。

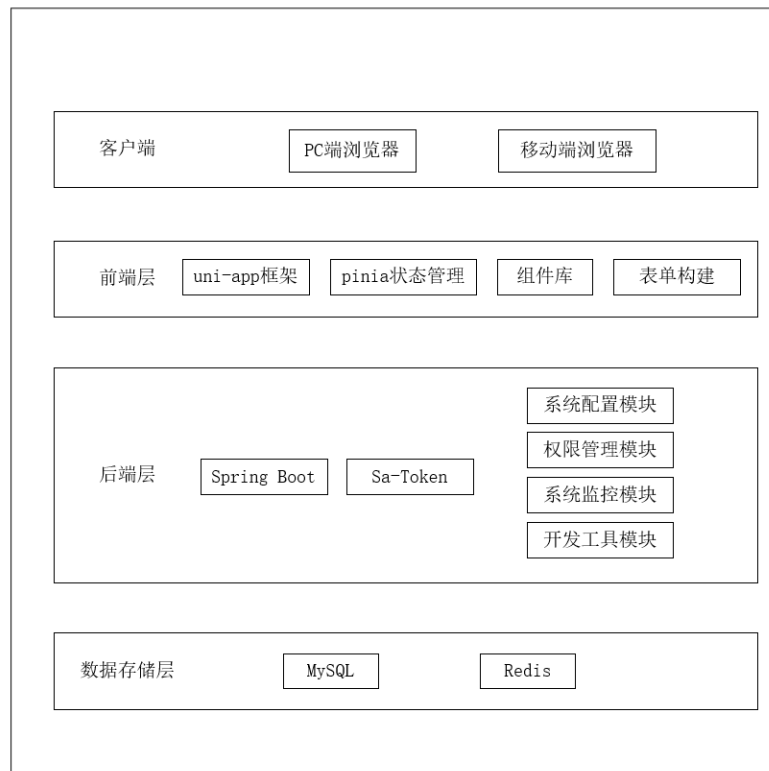


图 4-1 系统逻辑架构图

4.2 系统功能模块划分

根据第三章的需求分析结果，结合实际的 Web 应用开发场景，系统的功能模块划分如图 4-2 所示。

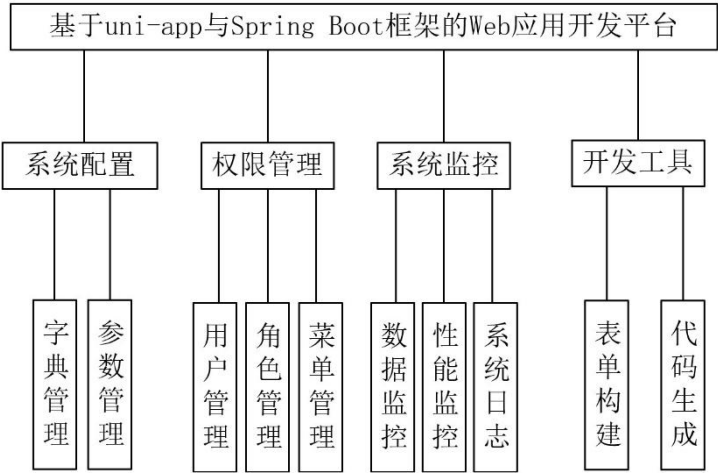


图 4-2 系统功能模块图

系统共划分为四个功能模块，分别是系统配置模块、权限管理模块、系统监控模块、开发工具模块。系统配置模块用于提供动态的参数管理与字典数据维护功能，包括字典管理和参数管理。权限管理模块用于登录认证和权限管理，主要包括用户管理、角色管理和菜单管理。系统监控模块用于实时展示系统运行状态和管理日志记录，包括数据监控、性能监控和日志监控。开发工具模块用于可视化生成前后端代码，包括表单构建功能和代码生成功能。

4.3 数据库设计

基于需求分析结果，本节首先通过实体-联系图（E-R 图）构建数据库概念模型，然后依据数据库规范化理论完成系统的表结构设计。

4.3.1 概念模型设计

字典管理模块的 E-R 模型如图 4-3 所示。

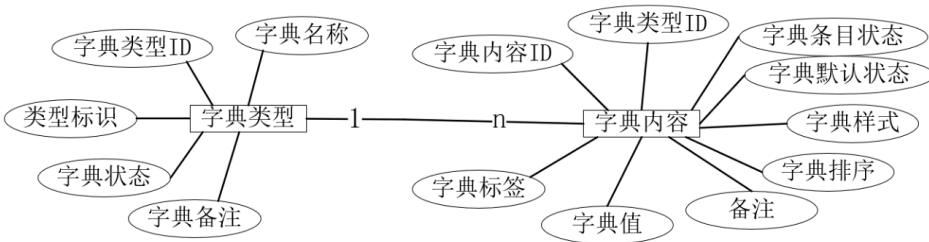


图 4-3 字典管理 E-R 图

参数管理模块的 E-R 模型如图 4-4 所示。

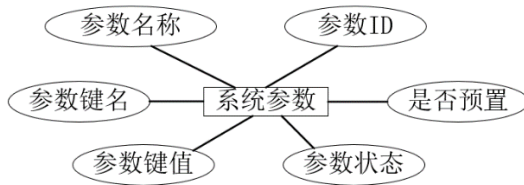


图 4-4 参数管理 E-R 图

权限管理模块的 E-R 模型如图 4-5 所示。

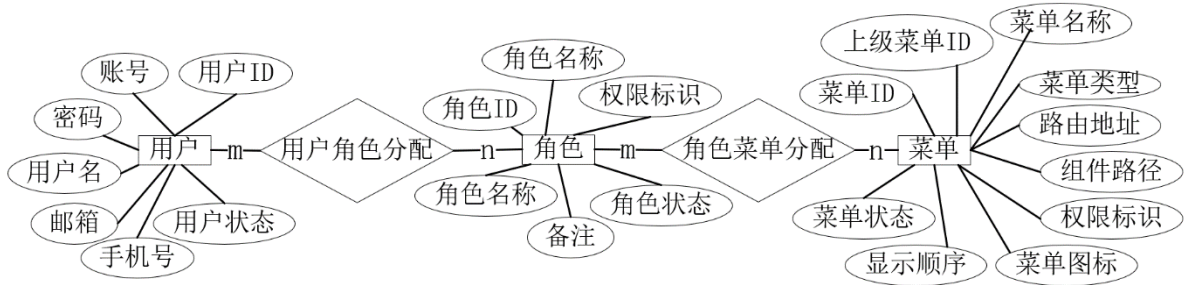


图 4-5 权限管理 E-R 图

日志管理模块的 E-R 模型如图 4-6 所示。

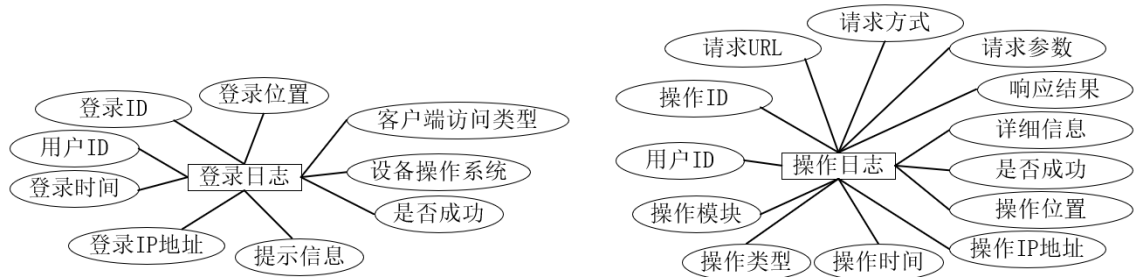


图 4-6 日志管理 E-R 图

代码生成模块的 E-R 模型如图 4-7 所示。

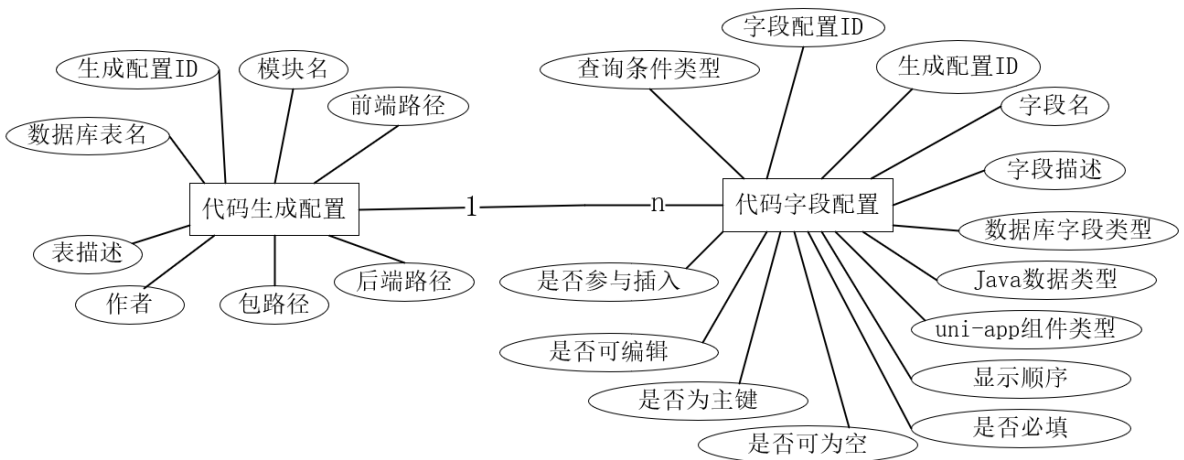


图 4-7 代码生成 E-R 图

4.3.2 数据库表设计

本节根据 E-R 模型设计数据库表结构。除了 E-R 图定义的数据和关系之外，为满足规范化系统对操作责任追溯、异常排查和风险管理等要求^[40]，所有需要追踪数据变更记录的表，须增加创建人、创建时间、更新人、更新时间、逻辑删除标志五个审计字段。

本系统一共设计 12 张表，分别为字典类型表、字典内容表、系统参数表、用户表、角色表、菜单表、用户角色关联表、角色菜单关联表、登录日志表、操作日志表、代码生成配置表、代码生成字段表。

(1) 字典类型表

字典类型表（sys_dict_type）用于存储字典类型数据，主要字段包括字典类型 ID、字典类型、字典名称、字典类型状态、备注。表的核心结构及关键字段设计如表 4-1 所示。

表 4-1 字典类型表

字段名	类型	约束	描述
dict_id	bigint unsigned	主键自增	字典类型 ID
dict_type	varchar(50)	非空，唯一	字典类型
dict_name	varchar(50)	非空	字典名称
is_active	unsigned tinyint	默认值为 0	字典类型状态（1-启用 0-停用）
remark	varchar(255)		备注

(2) 字典内容表

字典内容表（sys_dict_data）用于存储每个字典类型的所有字典内容数据，主要字段包括字典内容 ID、字典类型、字典标签、字典值、字典条目状态、默认状态、字典排序、条目展示样式、备注。表的核心结构及关键字段设计如表 4-2 所示。

表 4-2 字典内容表

字段名	类型	约束	描述
data_id	bigint unsigned	主键自增	字典内容 ID
dict_type	varchar(50)	非空，外键	字典类型
data_label	varchar(100)	非空	字典标签
data_value	varchar(100)	非空	字典值
is_active	unsigned tinyint	默认值为 0	字典条目状态（1-启用 0-停用）
is_default	unsigned tinyint	默认值为 0	默认状态（1-默认 0- 非默认）
data_sort	int	默认值为 0	字典排序
display_styles	json		条目展示样式
remark	varchar(255)		备注

(3) 系统参数表

系统参数表(sys_param)用于存储系统中可动态配置的参数,主要字段包括参数 ID、参数名称、参数键名、参数键值、参数状态、系统预置标志、备注。表的核心结构及关键字段设计如表 4-3 所示。

表 4-3 系统参数表

字段名	类型	约束	描述
param_id	bigint unsigned	主键自增	参数 ID
param_name	varchar(50)	非空	参数名称
param_key	varchar(100)	非空	参数键名
param_value	varchar(500)	非空	参数键值
is_active	unsigned tinyint	默认值为 1	参数状态 (1-启用 0-停用)
is_sys_builtin	unsigned tinyint	默认值为 0	系统预置 (1-是 0-否)
remark	varchar(255)		备注
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(4) 用户表

用户表 (sys_user) 主要用于存储用户的所有基础数据,主要字段包括用户 ID、用户账号、加密后的登录密码、用户名、邮箱地址、账号状态、逻辑删除标志、系统管理员标志。表的核心结构及关键字段设计如表 4-4 所示。

表 4-4 用户表

字段名	类型	约束	描述
user_id	bigint unsigned	主键自增	用户 ID
username	varchar(50)	非空, 唯一	登录账号
password	varchar(255)	非空	加密后的密码
nickname	varchar(50)	非空	显示名称
email	varchar(100)		邮箱地址
phone	varchar(20)		手机号码
is_active	unsigned tinyint	默认值为 1	账号状态 (1-启用 0-停用)
is_deleted	unsigned tinyint	默认值为 0	逻辑删除 (1-是 0-否)
is_admin	unsigned tinyint	默认值为 0	系统管理员 (0 否 1 是)
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(5) 角色表

角色表 (sys_role) 用于存储角色的所有基础数据,主要字段包括角色 ID、角色名称、角色权限标识、备注。表的核心结构及关键字段设计如表 4-5 所示。

表 4-5 角色表

字段名	类型	约束	描述
role_id	bigint unsigned	主键自增	角色 ID
role_name	varchar(50)	非空, 唯一	角色名称
role_key	varchar(100)	非空	角色权限标识
remark	varchar(500)		备注
is_active	unsigned tinyint	默认值为 1	角色状态(1-启用 0-停用)
is_deleted	unsigned tinyint	默认值为 0	逻辑删除标志(1-是 0-否)
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(6) 菜单表

菜单表(sys_menu)用于存储菜单的所有基础数据,主要字段包括菜单 ID、上级菜单 ID、菜单名称、菜单类型、路由路径、组件路径、权限标识、菜单图标、显示顺序、显示状态。表的核心结构及关键字段设计如表 4-6 所示。

表 4-6 菜单表

字段名	类型	约束	描述
menu_id	bigint unsigned	主键自增	菜单 ID
parent_id	bigint unsigned	默认值为 0	上级菜单 ID
name	varchar(50)	非空	菜单名称
type	tinyint	非空	菜单类型
path	varchar(200)		路由路径
component	varchar(255)		组件路径
perms	varchar(100)		权限标识
icon	varchar(100)		菜单图标
order_num	int	默认值为 0	显示顺序
is_visible	unsigned tinyint	默认值为 1	显示状态(1-显示 0-隐藏)
is_deleted	unsigned tinyint	默认值为 0	逻辑删除标志(1-是 0-否)
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(7) 用户角色关联表

用户角色关联表(sys_user_role)用于记录用户与角色之间的关联关系。主要字段包括用户 ID、角色 ID。表的核心结构及关键字段设计如表 4-7 所示。

表 4-7 用户角色关联表

字段名	类型	约束	描述
user_role_id	bigint unsigned	主键自增	用户角色关联 ID
user_id	bigint unsigned	外键，联合唯一索引	用户 ID
role_id	bigint unsigned	外键，联合唯一索引	角色 ID
is_deleted	unsigned tinyint	默认值为 0	逻辑删除（1-是 0-否）
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(8) 角色菜单关联表

角色菜单关联表（sys_role_menu）用于记录角色与菜单之间的关联关系。主要字段包括角色 ID、菜单 ID。表的核心结构及关键字段设计如表 4-8 所示。

表 4-8 角色菜单关联表

字段名	类型	约束	描述
role_menu_id	bigint unsigned	主键自增	角色菜单关联 ID
role_id	bigint unsigned	外键，联合唯一索引	角色 ID
menu_id	bigint unsigned	外键，联合唯一索引	菜单 ID
is_deleted	unsigned tinyint	默认值为 0	逻辑删除（1-是 0-否）
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

(9) 登录日志表

登录日志表（sys_login_log）用于存储用户登录的相关数据。主要字段包括登录 ID、用户 ID、登录时间、登录 IP 地址、登录位置、访问客户端类型、设备操作系统、成功标志、提示信息。表的核心结构及关键字段设计如表 4-9 所示。

表 4-9 登录日志表

字段名	类型	约束	描述
login_Id	bigint unsigned	主键自增	登录 ID
user_id	bigint unsigned	非空，外键	用户 ID
login_time	datetime	非空	登录时间
ipaddr	varchar(45)	非空	登录 IP 地址
location	varchar(100)		登录位置
browser	varchar(50)		访问客户端类型
os	varchar(50)		设备操作系统
is_success	unsigned tinyint	非空	是否成功（1-是 0-否）
msg	text		提示信息

(10) 操作日志表

操作日志表（sys_oper_log）用于存储用户操作的相关数据，主要字段包括操作 ID、

操作用户 ID、操作模块、操作类型、操作时间、操作 IP 地址、登录位置、操作状态、详细信息、请求 URL、请求方式、请求参数、响应结果。表的核心结构及关键字段设计如表 4-10 所示。

表 4-10 操作日志表

字段名	类型	约束	描述
oper_id	bigint unsigned	主键自增	操作 ID
user_id	bigint unsigned	非空, 外键	操作用户 ID
module	varchar(50)	非空	操作模块
type	varchar(50)		操作类型
oper_time	datetime	非空	操作时间
ipaddr	varchar(45)	非空	操作 IP 地址
location	varchar(100)		登录位置
is_success	unsigned tinyint	非空	是否成功 (1-是 0-否)
msg	text		详细信息
request_url	varchar(255)	非空	请求 URL
request_method	varchar(10)	非空	请求方式
request_params	text		请求参数
response_result	text		响应结果

(11) 代码生成配置表

代码生成配置表 (gen_table) 用于存储代码文件的基础配置数据, 主要字段包括配置 ID、数据库表名、表描述、作者、生成包路径、模块名、前端生成路径、后端生成路径、其他选项。表的核心结构及关键字段设计如表 4-11 所示。

表 4-11 代码生成配置表

字段名	类型	约束	描述
table_id	bigint unsigned	主键自增	配置 ID
table_name	varchar(200)	非空, 唯一	数据库表名
table_comment	varchar(500)		表描述
author	varchar(100)	非空	作者
package_name	varchar(200)	非空	生成包路径
module_name	varchar(100)	非空	模块名
frontend_path	varchar(200)		前端生成路径
backend_path	varchar(200)		后端生成路径
options	varchar(500)		其他选项 (JSON 格式)
is_deleted	unsigned tinyint	默认值为 0	逻辑删除 (1-是 0-否)

(12) 代码生成字段表

代码生成字段表(gen_field)用于存储字段的映射规则和交互逻辑, 主要字段包括字典 ID、关联表 ID、字段名称、数据库字段类型、字段描述、是否主键、是否可为空、Java 数据类型、uni-app 组件类型、是否必填、是否参与插入、是否可编辑、是否列表显示、查询条件类型、字段显示顺序。表的核心结构及关键字段设计如表 4-12 所示。

表 4-12 代码生成字段表

字段名	类型	约束	描述
field_id	bigint	主键自增	字段 ID
table_id	bigint	外键	关联表 ID
field_name	varchar(100)		字段名称
field_type	varchar(50)		数据库字段类型
field_comment	varchar(500)		字段描述
is_primary	tinyint(1)		是否主键
is_nullable	tinyint(1)		是否可为空
java_type	varchar(50)		Java 数据类型
frontend_type	varchar (100)		uni-app 组件类型
is_required	unsigned tinyint		是否必填
is_insert	unsigned tinyint		是否参与插入
is_edit	unsigned tinyint		是否可编辑
is_list	unsigned tinyint		是否列表显示
query_type	varchar(50)		查询条件类型
field_order	int		字段显示顺序
is_deleted	unsigned tinyint	默认值为 0	逻辑删除（1-是 0-否）
created_by	bigint unsigned	非空	创建人
created_time	datetime	非空	创建时间
updated_by	bigint unsigned		更新人
updated_time	datetime	非空	更新时间

4.4 本章小结

本章从实际的应用需求出发，完成了系统的总体设计，系统采用前后端分离架构，将系统划分为客户端层、前端层、后端层与数据存储层。通过功能模块化设计，将系统划分为系统配置、权限管理、系统监控、开发工具四个功能模块，并基于 E-R 图完成了数据库表结构设计，为后续开发奠定基础。

第五章 系统的详细设计与实现

5.1 系统开发环境与项目搭建

5.1.1 系统开发环境

系统基于前后端分离的开发模式，前端和后端是两个独立的工程。基础开发环境包括 JDK 和 Node，使用 Maven 和 npm 作为系统的包管理器，使用 HBuilderX 作为前端开发工具，前端基于 uni-app 框架开发，使用 Pinia 管理全局状态数据，使用 IntelliJ IDEA 作为后端开发工具，后端基于 Spring Boot 框架开发，使用 Navicat 管理 MySQL 数据库，使用 Redis 缓存系统的热点数据。使用 FreeMarker 作为代码生成的模板引擎。使用 Sa-Token 用于系统的登录认证、菜单授权和会话管理，使用 MyBatis-Plus 作为数据库操作框架，使用 JMeter 和 Postman 作为系统测试工具。系统开发环境如表 5-1 所示。

表 5-1 系统开发环境

分类	技术/工具/版本
前端技术	uni-app 3.8.12、vue.js 3.2.47、pinia 2.0.36
后端技术	Spring Boot 3.1.0、Sa-Token 1.39.0、 MyBatis-Plus 3.5.2、FreeMarker 2.3.32
数据存储工具	MySQL 8.0.34、Redis 6.2.12、Navicat 16
包管理工具	Maven 3.8.6、npm 8.19.4
基础开发环境	JDK 17.0.8、node 16.20.2
前后端开发工具	IntelliJ IDEA 2023.2、HBuilderX 4.45
测试工具	JMeter 5.6.2、Postman 10.18.0

5.1.2 项目的搭建

在 HBuilderX 中，创建项目的前端工程，选择 uni-ui 项目模板，创建项目名称为 DevFace 的工程目录。其中 pages 目录用于存放前端的页面文件，api 目录用于存放向后端发起请求的所有接口文件，components 目录用于存放所有可复用的公共组件，store 目录用于存放系统的全局状态管理文件，utils 目录用于存放开发人员封装或编写的工具函数，uni.scss 文件用于统一定义全局的样式变量，pages.json 文件用于配置系统路由与页面的样式，manifest.json 是 uni-app 的全局配置文件。系统的前端目录结构如图 5-1 所示。



图 5-1 前端目录结构图

在 IntelliJ IDEA 中，创建项目的后端工程名为 DevCore，系统的后端基于 maven 多模块结构，其中 devcore-common 模块是基础模块，用于存放全局工具类、自定义注解和统一的异常处理类等通用类，被其他所有模块依赖。devcore-web 模块是项目启动模块，主要用于运行程序和处理所有接口请求。devcore-auth 是认证授权模块，负责用户登录、Token 签发、权限校验等功能。devcore-system 是系统的核心业务模块。devcore-monitor 是系统监控模块。devcore-developer 是开发工具模块。系统后端目录结构如图 5-2 所示。

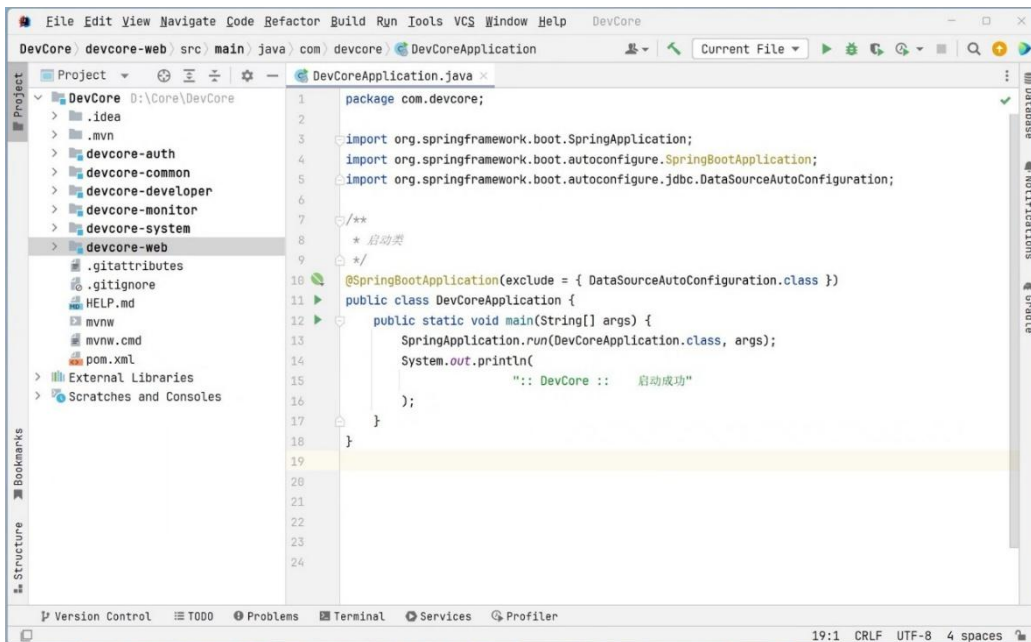


图 5-2 后端目录结构图

5.2 系统配置模块

系统配置模块为用户提供统一的配置管理服务，包括字典管理和参数管理。

5.2.1 字典管理

字典管理模块主要为用户提供字典管理的服务，主要包括对字典类型的管理以及对每个字典类型包含的所有字典内容进行管理。

字典类型管理模块包括对字典类型的增删改查等功能，主要涉及的类包括 DictTypeController、DictTypeService、DictTypeForm、DictTypeQueryVo。

字典管理的主要接口如表 5-2、表 5-3 所示。

表 5-2 字典类型管理模块接口

接口	请求方式	请求参数	返回参数	描述
/dict-types	GET	DictTypeQuery	Result<PageResult<DictTypeVO>>	分页查询
/dict-types	POST	DictTypeForm	Result<Void>	新增字典
/dict-types/{id}	PUT	DictTypeUpdateForm	Result<Void>	修改字典
/dict-types/{id}	DELETE	无	Result<Void>	删除字典
/dict-types	DELETE	BatchDeleteDTO	Result<Void>	批量删除
/dict-types/cache	PATCH	无	Result<Void>	刷新缓存

表 5-3 字典内容管理模块接口

接口	请求方式	请求参数	返回参数	描述
/dict-data	GET	DictDataQuery	Result<PageResult<DictDataVO>>	分页查询
/dict-data	POST	DictDataForm	Result<Void>	新增字典内容
/dict-data/{id}	PUT	DictDataUpdateForm	Result<Void>	修改字典内容
/dict-data/{id}	DELETE	无	Result<Void>	删除字典内容
/dict-data	DELETE	BatchDeleteDTO	Result<Void>	批量删除

字典管理界面如图 5-3、图 5-4 所示。



图 5-3 字典类型管理界面



图 5-4 字典内容管理界面

(1) 查询字典类型

用户在前端页面的查询表单中，输入查询条件，点击“查询”按钮。前端将查询条件封装为 DictTypeQuery 对象，发起 GET 请求至/dict-types 接口。DictTypeController 接收请求后调用 DictTypeService 的 getDictTypeList()方法，通过 DictTypeMapper 查询数据库中的 sys_dict_type 表，返回分页结果 PageResult<DictTypeVO>，前端以表格的形式展示数据。

(2) 新增字典类型

用户点击“新增”按钮，填写字典名称、字典类型等信息。前端将表单数据封装为 DictTypeForm 对象，发送 POST 请求至/dict-types 接口。DictTypeService 校验字典类型唯一性后，调用 DictTypeMapper 插入 sys_dict_type 表，并记录操作人及操作时间。

(3) 批量删除字典类型

用户勾选多行数据后点击“批量删除”按钮，前端发送携带 BatchDeleteDTO 的 DELETE 请求至/dict-types 接口。后端根据 id 列表逻辑删除 sys_dict_type 记录，同时级联删除关联的字典内容。触发 PATCH 请求至/dict-types/cache，后端调用 Redis 服务清空字典缓存并重新加载最新数据。

5.2.2 参数管理

参数管理模块为用户提供系统参数的动态配置服务，主要涉及的类包括 ParamController、ParamService、ParamMapper、ParamForm 及 ParamQueryVo。参数管理模块的主要接口如表 5-4 所示。

表 5-4 参数管理模块接口

接口	请求方式	请求参数	返回参数	描述
/params	GET	ParamQuery	Result<PageResult<ParamVO>>	分页查询
/param	POST	ParamForm	Result<Void>	新增参数
/param/{id}	PUT	ParamUpdateForm	Result<Void>	修改参数
/param/{id}	DELETE	无	Result<Void>	删除参数
/params	POST	ParamBatchDelDTO	Result<Void>	批量删除
/params/cache	PATCH	无	Result<Void>	刷新缓存

参数管理界面如图 5-5 所示。



图 5-5 参数管理界面

(1) 查询参数

用户在前端页面的查询表单中，输入参数名称或参数键名，点击“查询”按钮。前端通过 ParamQuery 封装查询条件，发送 GET 请求至/params 接口。ParamService 调用 ParamMapper 执行动态 SQL 查询 sys_param 表，返回 PageResult<ParamVO>。查询结果中的系统预置参数（is_sys_builtin=1）禁止前端删除或修改。

(2) 新增参数

用户在表单中填写参数键名、键值及备注，前端提交 ParamForm 至/param 接口。ParamService 校验键名唯一性后插入数据，若键名重复则返回错误提示。关键字段 param_key 采用唯一索引约束，确保数据一致性。

(3) 批量删除参数

批量删除时，ParamService 过滤系统预置参数，仅允许删除用户自定义参数。删除逻辑采用逻辑删除（is_deleted=1），参数修改或删除后，前端自动触发/params/cache 接口的 PATCH 请求。后端通过 RedisTemplate 删除原有缓存键，并重新加载全部参数至 Redis，保障参数管理实时生效。

5.3 权限管理模块

系统的权限管理模块基于 RBAC 模型，主要功能包括用户管理、角色管理、菜单管理。用户通过角色获取权限，角色拥有菜单的访问权限。

5.3.1 用户管理

用户管理模块主要包括用户登录功能和系统用户管理功能。

用户登录模块作为系统访问的入口，承担身份认证、会话管理及登录日志记录的安全职责。本模块基于 Sa-Token 无状态会话机制，使用 BCrypt 加密算法与多端登录控制策略，保障用户身份的安全性与系统访问的可靠性。

用户登录相关的主要接口如表 5-5 所示。

表 5-5 用户登录模块接口

接口	请求方式	请求参数	返回参数	描述
/auth/login	POST	UserLoginReq	Result<UserLoginRes>	用户登录
/auth/logout	DELETE	无	Result<Void>	退出登录
/users/{userId}/password	PUT	ChangePwdReq	Result<Void>	修改密码
/users/me	GET	无	Result<UserProfileRes>	获取当前用户信息

用户在登录页面输入账号与密码,前端基于 Vue.js 双向绑定机制实时校验输入格式。点击“登录”按钮后,前端通过 uni.request 发起 POST 请求至/auth/login 接口,请求体封装为 JSON 格式的 UserLoginReq 对象,后端服务接收请求并通过@Valid 注解对 UserLoginReq 进行基础校验,然后 UserService 调用 loadUserByUsername(username)方法查询 sys_user 表,比对 BCrypt 加密后的密码哈希值。若用户不存在或密码错误,抛出 ServiceException("账号或密码错误")。否则,校验用户状态(is_active 字段),若账号被停用则拒绝登录,并记录异常日志。

若登录成功,则通过 Sa-Token 的 StpUtil.login(userId)生成无状态 Token,登录成功后,后端返回 UserLoginRes 对象,包含 Token、用户基础信息及权限列表。前端将 Token 存储至 Pinia 状态库,并通过请求拦截器自动附加至后续请求头。异步写入登录日志:通过 Spring 事件机制发布 LoginEvent,由 LoginLogListener 监听并调用 LoginLogService 插入 sys_login_log 表,记录 IP 地址、设备信息及登录结果。

用户登录成功后,系统根据用户的角色查询所有关联的 perms 权限标识,存入 Token。用户发起请求时, Sa-Token 解析 Token 中的权限列表。用户访问页面时,前端校验用户所有角色的权限列表中是否存在路由对应的权限标识。后端校验接口注解中的权限标识是否在用户权限列表中。若未包含,拦截请求并返回 403 错误若不存在,则跳转至无权限页面。用户登录流程如图 5-6 所示。

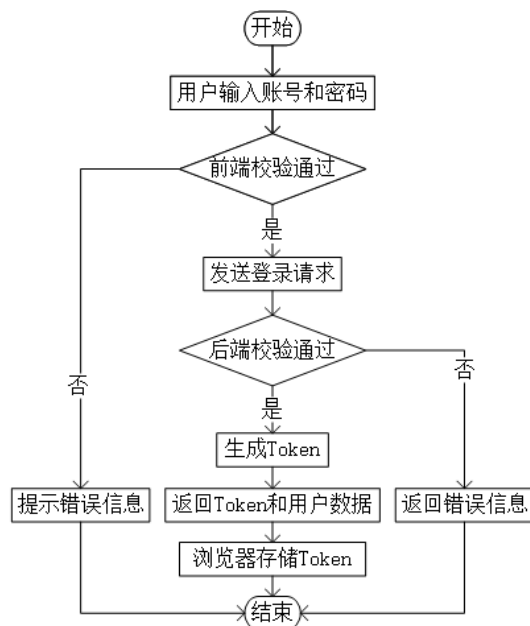


图 5-6 用户登录流程图

用户登录的界面实现如图 5-7 所示。



图 5-7 用户登录表单

用户管理模块用于维护系统用户的基础信息及角色分配，核心功能包括对用户的新增、删除、修改、条件查询、角色分配及状态管理，主要涉及 UserController、UserService、UserMapper、User 实体类及 UserQuery 查询参数类。基于 Spring Boot 与 Sa-Token 框架，通过 RBAC 模型实现用户与角色的动态关联。系统用户管理的主要接口如表 5-6 所示。

表 5-6 用户管理模块接口

接口	请求方式	请求参数	返回参数	描述
/users	GET	UserQuery	Result<PageResult<UserVO>>	分页查询
/users	POST	UserForm	Result<Void>	新增用户
/users/{id}	PUT	UserUpdateForm	Result<Void>	修改用户信息
/users/{id}	DELETE	无	Result<Void>	删除用户
/users	DELETE	BatchDeletedDTO	Result<Void>	批量删除
/users/{userId}/roles	GET	无	Result<UserRoleAllocationVO>	查询角色列表
/users/{userId}/roles	PUT	RoleFullUpdateDTO	Result<Void>	更新角色列表

用户管理界面实现如图 5-8 所示。



图 5-8 用户管理界面

（1）查询用户

在前端页面的查询表单中，输入用户账号或用户名等信息，点击“查询”按钮。前端将查询参数封装至 UserQuery 对象，调用 UserController 的 getUserList()接口。UserService 层解析参数后，通过动态 SQL 构建查询条件，利用 UserMapper 的 selectUserList()方法访问 sys_user 表，返回分页数据。查询结果包含用户基础信息及关联角色名称，前端以表格形式展示。

（2）新增用户

新增用户时，须填写用户账号、用户名等信息，可以选择关联的角色。前端通过 UserController 的 addUser()接口提交 UserForm 对象，UserService 层校验用户账号的唯一性后，对密码进行 BCrypt 加密，调用 UserMapper 的 insertUser()方法写入 sys_user 表。同时，通过 UserRoleService 将用户与角色绑定关系写入 sys_user_role 关联表，确保权限即时生效。

（3）修改用户

修改用户信息时，前端提交 UserUpdateForm 至 UserController 的 updateUser()接口。UserService 层校验数据合法性后，更新 sys_user 表记录。若涉及角色变更，通过事务管理同步更新 sys_user_role 表，避免权限数据不一致。支持重置密码功能，调用 SaToken 的 updatePassword()接口更新密码哈希值。

（4）删除用户

删除操作支持逻辑删除（标记 is_deleted 字段为 1）与批量执行。前端传递用户 id 列表至 UserController 的 deleteUser()接口，UserService 层校验用户是否关联业务数据，若无依赖则更新删除标志。账号停用/启用通过 UserController 的 updateUserStatus()接口实现，触发 SaToken 的踢人下线机制，强制目标用户重新认证。

（5）用户角色分配

通过 UserController 的 assignRoles()接口，前端提交用户 id 与角色 id 列表。UserRoleService 层先清空用户原有角色绑定，再批量插入新关联关系至 sys_user_role 表。

基于 SaToken 的权限缓存刷新机制，确保用户权限实时生效。用户角色分配的示例代码如下表 5-7 所示。

表 5-7 用户角色分配示例代码

```
// UserRoleService.java
@Transactional
public void assignRoles(Long userId, List<Long> roleIds) {
    // 清空用户原有角色
    userRoleMapper.deleteByUserId(userId);
    // 批量插入新角色关联
    List<UserRole> userRoles = roleIds.stream()
        .map(roleId -> new UserRole(userId, roleId))
        .collect(Collectors.toList());
    userRoleMapper.batchInsert(userRoles);
    // 刷新用户权限缓存
    SaTokenManager.refreshPermission(userId);
}
// SQL 映射（MyBatis-Plus）
<insert id="batchInsert">
    INSERT INTO sys_user_role (user_id, role_id) VALUES
    <foreach collection="list" item="item" separator=",">
        (#{item.userId}, #{item.roleId})
    </foreach>
</insert>
```

用户角色分配界面如图 5-9 所示。

☐	角色ID	角色账号	角色标识	创建时间
☐	1001	系统管理员	admin	2024-11-18 10:51:27
☐	1002	测试角色	test1	2024-11-18 10:51:27
☐	1003	测试角色2	test2	2024-11-18 10:51:27

共 3 条

10条/页

< 1 >

前往 1 页

确定

返回

图 5-9 用户角色分配界面

5.3.2 角色管理

角色管理模块用于维护系统角色的基础信息及其与用户和菜单之间的关联关系，主要功能包括角色的新增、删除、修改、条件查询、为角色分配用户、为角色关联菜单，主要涉及 RoleController、RoleService、RoleMapper、Role 实体类及 RoleQuery 的相关查询参数类。

角色管理模块的主要接口如表 5-8、表 5-9 所示。

表 5-8 角色管理模块接口

接口	请求方式	请求参数	返回参数	描述
/roles	GET	RoleQueryDTO	Result<PageResult<RoleVO>>	分页查询
/roles	POST	RoleCreateDTO	Result<Void>	新增角色
/roles/{id}	PUT	RoleUpdateDTO	Result<Void>	修改角色
/roles/{id}	DELETE	无	Result<Void>	删除角色
/roles	DELETE	RoleBatchDeleteDTO	Result<Void>	批量删除
/roles/{id}/status	PATCH	RoleStatusUpdateDTO	Result<Void>	修改状态

表 5-9 角色分配主要接口

接口	请求方式	请求参数	返回参数	描述
/roles/{roleId}/users	GET	UserQuery	Result<PageResult<UserVO>>>	分页查询
/roles/{roleId}/users	POST	RoleUserRelationDTO	Result<Void>	批量关联
/roles/{roleId}/users	DELETE	RoleUserRelationDTO	Result<Void>	批量取消

角色管理界面如图 5-10 所示。



图 5-10 角色管理界面

(1) 查询角色

角色管理模块用于维护系统角色的基础信息及其与用户、菜单的关联关系，主要功能包括角色的新增、删除、修改、条件查询，主要涉及 RoleController、RoleService、RoleMapper、Role 实体类及 RoleQuery 查询参数类。系统通过分层架构实现业务逻辑与数据操作的解耦，保障系统的可维护性与扩展性。

用户可通过角色名称或权限标识进行条件查询。前端将输入的查询条件封装至 RoleQueryDTO 对象，调用 RoleController 的 listRoles()接口。RoleService 层接收请求后，基于 RoleVo 构建动态查询条件，通过 RoleMapper 的 listRoles()方法访问数据库 sys_role 表，获取符合条件的数据集并返回至前端。返回结果以分页形式展示，支持按角色状态筛选，确保查询效率。

(2) 新增角色

新增角色时，用户需填写角色名称、权限标识及备注信息。前端将表单数据封装为 Role 实体对象，调用 RoleController 的 createRole()接口。RoleService 层校验角色名称唯一性后，通过 RoleMapper 的 createRole()方法将数据持久化至 sys_role 表。同时，通过权

限标识（role_key）与菜单权限的绑定，实现角色权限的动态配置，确保数据一致性。角色新增界面如图 5-11 所示。

The figure shows a web form for adding a new role. The form is titled "新增角色" (Add Role) and includes a close button (X). The form contains the following fields and options:

- * 角色名称** (Role Name): A text input field with the placeholder "请输入角色名称" (Please enter role name).
- * 权限标识** (Permissions): A text input field with the placeholder "请输入权限字符" (Please enter permission character).
- 关联菜单** (Associated Menu): A section with two radio buttons: "全选" (Select All) and "级联勾选" (Cascade Selection). Below these is a tree view of menu items:
 - ☒ 配置管理 (Configuration Management)
 - ☒ 权限管理 (Permissions Management)
 - ☐ 系统监控 (System Monitoring)
 - ☒ 系统工具 (System Tools)
 - ☒ uni-app 表单构建 (uni-app Form Builder)
 - ☐ 模板代码生成 (Template Code Generation)
- 角色状态** (Role Status): Two radio buttons: "正常" (Normal) and "停用" (Disabled).
- * 显示排序** (Display Order): A numeric input field with the value "2" and up/down arrows.
- 备注** (Remarks): A text area with the placeholder "请输入内容" (Please enter content).
- Buttons**: "确定" (Confirm) and "取消" (Cancel) buttons at the bottom right.

图 5-11 角色新增界面

（3）修改角色

修改角色信息时，用户选择目标角色并更新角色名称、权限标识、关联的菜单等信息。前端将修改后的数据提交至 RoleController 的 updateRole() 接口，RoleService 层验证权限标识冲突后，调用 RoleMapper 的 updateRole() 方法更新数据库记录。若角色已关联用户或菜单，系统通过事务管理确保关联数据的同步更新，避免脏数据产生。

（4）删除角色

删除操作支持单条删除或批量执行。前端传递角色 ID 列表至 RoleController 的 deleteRole() 接口，RoleService 层校验角色是否被用户或菜单引用。若存在关联数据，则提示“角色已被使用，不可删除”；若无关联，则通过 RoleMapper 的 deleteRoleByIds() 方法逻辑删除数据库记录（标记 is_deleted 字段为 1），保障数据可追溯性。

（5）角色用户分配

通过 RoleController 的 assignUsers() 接口，前端提交角色 id 与用户 id 列表至后端。RoleUserService 层先清空角色原有用户绑定，再批量插入新用户关联关系至 sys_user_role 表。基于 SaToken 的权限缓存刷新机制，确保用户权限实时生效。为角色分配用户的示例代码如下表 5-10 所示。

表 5-10 角色用户分配示例代码

```
// RoleUserService.java
@Transactional
public void assignUsers(Long roleId, List<Long> userIds) {
    // 清空角色原有用户
    userRoleMapper.deleteByRoleId(roleId);
    // 批量插入新用户关联
    List<UserRole> userRoles = userIds.stream()
        .map(userId -> new UserRole(userId, roleId))
        .collect(Collectors.toList());
    userRoleMapper.batchInsert(userRoles);
    // 刷新相关用户权限缓存
    userIds.forEach(SaTokenManager::refreshPermission);
}
```

角色用户分配界面如图 5-12 所示。

☐	用户账号	用户名	状态	创建时间	操作
☐	admin	系统管理员	正常	2024-11-18 10:51:27	取消关联
☐	test1	测试用户1	正常	2024-11-18 10:51:27	取消关联

共 2 条 10条/页 < 1 > 前往 1 页

图 5-12 角色用户分配界面

(6) 角色菜单分配

在新增和修改角色时，可管理角色与菜单之间的关联关系。通过 RoleMenuService 的 assignMenus 接口，先清空角色原有菜单，然后递归获取所有子菜单 ID，在批量插入关联后，刷新角色权限缓存。角色菜单分配示例代码如表 5-11 所示。

表 5-11 角色菜单分配示例代码

```
// RoleMenuService.java
@Transactional(rollbackFor = Exception.class)
public void assignMenus(Long roleId, List<Long> menuIds) {
    // 清空角色原有菜单
    roleMenuMapper.deleteByRoleId(roleId);
    // 递归获取所有子菜单 ID（包含所有层级）
    Set<Long> allMenuIds = getMenuTreeIds(menuIds);
    // 批量插入关联
    List<RoleMenu> roleMenus = allMenuIds.stream()
        .map(menuId -> new RoleMenu(roleId, menuId))
        .collect(Collectors.toList());
}
```

表 5-11 角色菜单分配示例代码（续）

```
roleMenuMapper.batchInsert(roleMenus);
// 刷新角色权限缓存
SaTokenManager.refreshRole(roleId);}
// 获取所有层级的菜单 ID
public Set<Long> getMenuTreeIds(List<Long> menuIds) {
    Set<Long> result = new HashSet<>();
    for (Long menuId : menuIds) {
        // 递归添加当前菜单及其所有子菜单
        addMenuAndChildren(menuId, result);}return result;}
private void addMenuAndChildren(Long menuId, Set<Long> result) {
    // 1. 添加当前菜单 ID
    result.add(menuId);
    // 2. 查询直接子菜单（使用自定义的 selectByParentId 方法）
    List<Menu> children = menuMapper.selectByParentId(menuId);
    // 3. 递归处理每个子菜单
    for (Menu child : children) {Long childId = child.getMenuId();
        if (!result.contains(childId)) {addMenuAndChildren(childId,
result);}}}
```

5.3.3 菜单管理

菜单管理模块用于维护系统菜单之间的层级关系与权限标识。通过树表格可视化展示菜单关系，菜单可分为目录、页面和按钮。

菜单管理主要涉及的类包括 MenuController、MenuServiceImpl、MenuMapper、MenuTreeVO 及 MenuForm，采用邻接表结构存储树形关系，支持动态权限渲染与可视化层级管理。菜单管理的主要接口如表 5-12 所示。

表 5-12 菜单管理主要接口

接口	请求方式	请求参数	返回参数	描述
/menus	GET	MenuQuery	Result<PageResult<MenuVO>>	分页查询
/menus	POST	MenuForm	Result<Void>	新增菜单
/menus/{id}	PUT	MenuUpdateForm	Result<Void>	修改菜单
/menus/{id}	DELETE	无	Result<Void>	删除菜单
/menus	DELETE	BatchDeleteDTO	Result<Void>	批量删除

（1）查询菜单

用户访问菜单管理页面时，前端发送 GET 请求至/menus 接口。MenuController 接收请求后调用 MenuService 类的 getMenuTree()方法，通过 MenuMapper 执行 SQL 查询 sys_menu 表。查询结果封装为 PageResult<MenuVO>对象，前端使用树表格组件递归渲染树形结构数据。节点的展开/折叠状态通过 expandedKeys 字段维护，全选/半选逻辑基于父子级联关系动态计算。菜单管理界面如图 5-13 所示。

菜单名称

菜单状态

查询

重置

新增

菜单名称	权限标识	组件路径	菜单状态	创建时间	相关操作
配置管理			正常	2024-12-10 10:18:45	修改 新增 删除
参数管理	param:config:view	param/config/index	正常	2024-12-10 10:18:45	修改 新增 删除
参数查询	param:config:query		正常	2024-12-10 10:18:45	修改 新增 删除
参数新增	param:config:add		正常	2024-12-10 10:18:45	修改 新增 删除
参数修改	param:config:edit		正常	2024-12-10 10:18:45	修改 新增 删除
参数删除	param:config:del		正常	2024-12-10 10:18:45	修改 新增 删除
字典管理	dict:type:view	dict/type/index	正常	2024-12-10 10:18:45	修改 新增 删除
权限管理			正常	2024-12-10 10:18:45	修改 新增 删除
系统监控			正常	2024-12-10 10:18:45	修改 新增 删除
系统工具			正常	2024-12-10 10:18:45	修改 新增 删除

图 5-13 菜单管理界面

(2) 新增菜单

用户点击"新增"按钮，选择菜单类型（目录/页面/按钮），填写菜单名称、路由地址等数据。前端将表单数据封装为 MenuForm 对象，发送 POST 请求至/menus 接口。后端 MenuService 调用 checkMenuKeyUnique(perms)方法验证权限标识全局唯一。验证后通过 MenuMapper 向 sys_menu 表插入记录，自动填充父节点 id 及 order_num，同时记录操作人和操作时间。新增菜单界面如图 5-14 所示。

新增菜单

上级菜单

选择上级菜单

根目录 (4)

配置管理 (2)

参数管理 (4)

字典管理 (6)

权限管理 (3)

系统监控 (6)

系统工具 (2)

菜单类型

☒ 目录 ☐ 页面 ☐ 按钮

* 显示排序

1

* 菜单名称

请输入菜单名称

* 路由地址

请输入路由地址

目录状态

☒ 正常 ☐ 停用

确定

取消

图 5-14 新增菜单界面

（3）修改菜单

用户选中菜单节点后点击“修改”按钮，前端回显表单数据。提交 MenuUpdateForm 至 /menus/{id} 接口，MenuService 通过动态 SQL 更新非空字段。前端监听树节点的 @node-drop 事件，计算节点新位置后提交包含 parent_id（新父 ID）和 order_num（新序号）的 MenuSortDTO 对象至 /menus/sort 接口。后端批量执行 UPDATE sys_menu SET parent_id=#{parentId}, order_num=#{orderNum} WHERE menu_id=#{menuId}，保障事务一致性。

（4）删除菜单

用户选中菜单节点后点击“删除”按钮时，前端弹出二次确认对话框，确认后发送 DELETE 请求至 /menus/{id} 接口。后端执行删除操作前进行如下校验：首先调用 existsSubMenu 检查是否存在子菜单，若有则提示“存在子菜单，不可删除”；然后通过 roleMenuMapper.existsByMenuId 验证是否被角色关联，已分配时提示“菜单已分配给角色，不可删除”。所有校验通过后执行删除操作，同时记录操作日志。删除过程中采用 Spring 事务管理确保数据一致性，

5.4 系统监控模块

系统监控模块作为平台运维能力的支撑，通过采集系统运行状态数据、实时查看服务性能指标及记录操作日志，为开发团队提供运维决策依据。本模块围绕数据监控、性能监控与系统日志三个子模块展开设计，结合轻量化架构特性，在保障功能完备性的同时降低资源消耗。

5.4.1 数据监控

Druid Monitor 是 Druid 连接池内置的监控组件，提供了对 SQL 执行状况、URI 请求统计等关键指标的实时监控功能，本平台通过集成该模块页面，为系统提供了多维度的运行时数据分析能力。

首先在后端项目中引入了 Druid 依赖，然后在配置文件中打开监控组件并配置 IP 白名单，然后在前端页面中引入该文件，具有该菜单访问权限的用户即可通过默认账号和密码查看实时数据信息。

数据监控页面如图 5-15 所示。

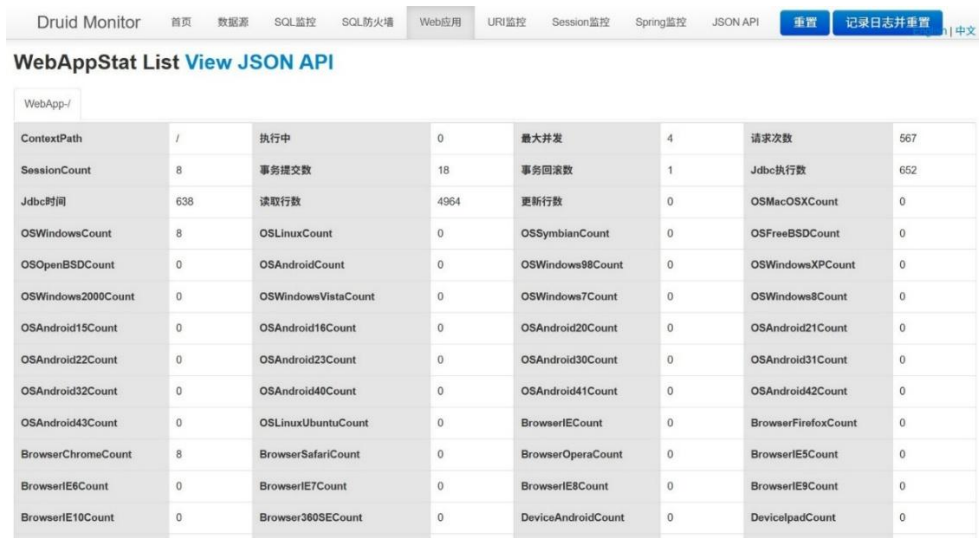


图 5-15 数据监控界面

5.4.2 性能监控

本模块基于 oshi 库实现系统资源数据采集, 通过 SystemInfo 获取硬件与操作系统信息, 通过 RedisTemplate 实现采集缓存数据, 通过 Echarts 绘图图表。性能监控的主要接口如表 5-13 所示。

表 5-13 性能监控主要接口

接口	请求方式	请求参数	返回参数	描述
/monitor/server	GET	ServerMonitorQuery	Result<T>	获取服务器监控数据
/monitor/cache	GET	CacheMonitorQuery	Result<Void>	获取 Redis 监控数据

性能监控界面通过仪表盘展示了系统硬件、JVM 以及 Redis 的各项指标。鼠标悬浮在仪表盘可查看详细信息。性能监控界面如图 5-16 所示^[41]。



图 5-16 性能监控界面

5.4.3 系统日志

系统日志模块包括登录日志管理和操作日志管理。

登录日志模块用于记录用户登录系统的详细信息，包括登录时间、IP 地址、设备类型及登录结果等，主要功能包括日志查询、批量删除及详情查看，主要涉及 LoginLogController、LoginLogService、LoginLogMapper、LoginLog 实体类及 LoginLogQueryVo 查询参数类。登录日志的主要接口如表 5-14 所示。

表 5-14 登录日志主要接口

接口	请求方式	请求参数	返回参数	描述
/login-logs	GET	LoginLogQuery	Result<PageResult<LoginLogVO>>	分页查询
/login-logs	DELETE	BatchDeleteDTO	Result<Void>	批量删除
/login-logs/all	DELETE	无	Result<Void>	清空所有

(1) 查询登录日志

用户通过用户登录账号、IP 地址、登录时间等条件查询日志。前端将查询参数封装至 LoginLogQuery 对象，调用 LoginLogController 的 getLoginLogList()接口。LoginLogService 层解析查询条件，动态构建 SQL 语句，通过 LoginLogMapper 的 selectLoginLogList()方法访问 sys_login_log 表，返回分页数据。日志列表展示登录账号、时间、IP、设备及状态字段，支持按时间排序，提升排查效率。

(2) 批量删除与清空日志

用户可按条件筛选后删除或全量清空日志。前端传递日志 ID 列表至 LoginLogController 的 deleteLoginLog()接口，LoginLogService 层调用 LoginLogMapper 的 deleteLoginLogByIds()方法物理删除数据库记录。全量清空时，直接执行 TRUNCATE TABLE sys_login_log 语句，清空所有登录日志。

登录日志管理界面如图 5-17 所示。



图 5-17 登录日志管理界面

操作日志模块用于记录用户在系统各模块中的关键操作行为,通过 AOP 切面与异步写入技术,实现操作行为的无侵入式记录。主要功能包括查询操作日志、批量删除与查看详情,主要涉及 OperLogController、OperLogService、OperLogMapper、OperLog 实体类及 OperLogQueryVo 查询参数类。操作日志管理的主要接口如表 5-15 所示。

表 5-15 操作日志主要接口

接口	请求方式	请求参数	返回参数	描述
/operation-logs	GET	OperationLogQuery	Result<PageResult<Operati onLogVO>>	分页查询
/operation-logs	DELETE	BatchDeleteDTO	Result<Void>	批量删除
/operation-logs/all	DELETE	无	Result<Void>	清空所有
/operation-logs/{id}/detail	GET	无	Result<OperationDetailVO>	查看详情

(1) 查询操作日志

用户可按操作模块、操作类型等条件筛选日志。前端封装查询条件至 OperationLogQuery,调用 OperLogController 的 getOperLogList()接口。OperLogService 层通过 OperLogMapper 的 selectOperLogList()方法查询 sys_oper_log 表,返回分页数据。列表展示操作用户、操作模块、操作类型、操作时间等信息。

(2) 批量删除与清空操作日志

前端传递日志 ID 列表至 OperLogController 的 deleteOperLog()接口,OperLogService 层调用 OperLogMapper 的 deleteOperLogByIds()方法物理删除记录。支持按条件筛选后批量删除,或通过 TRUNCATE TABLE sys_oper_log 清空全表,释放存储空间。

(3) 查看操作日志的详细信息

点击日志列表中的“详细信息”按钮,前端调用 OperLogController 的 getOperLogDetail()接口,通过日志 ID 查询 sys_oper_log 表完整记录,包括请求参数、响应结果、异常堆栈等信息。数据通过 OperLogDetailVO 对象返回,前端以折叠面板形式展示,便于开发人员快速定位问题。

操作日志管理界面如图 5-18 所示。



图 5-18 操作日志管理界面

5.5 开发工具模块

开发工具模块提供了面向 uni-app 组件的表单构建功能和基于 FreeMarker 模板的代码生成功能。

5.5.1 表单构建

表单构建模块通过可视化拖拽与动态配置，实现前端页面组件的快速生成与代码导出，能有效降低重复开发工作量。本模块在 form-generator 开源项目的基础之上，基于 uni-app 框架的组件化特性以及 Vue.js 响应式数据绑定与 Pinia 状态管理，构建了一套低门槛、高效率的表单设计解决方案。

（1）功能架构设计

表单构建模块采用“组件库-设计器-配置面板”三层架构，支持用户从左侧组件库拖拽元素至画布区域，通过右侧配置面板调整属性，并实时预览生成效果。最终可通过代码导出功能生成符合 uni-app 规范的 Vue 单文件组件（SFC），覆盖表单布局、数据绑定及校验逻辑的全流程需求。

组件库层内置了基础表单组件与布局容器，每类组件封装为独立的 Vue 组件，遵循 props-in/events-out 设计原则。组件元数据通过 JSON 配置文件定义，便于动态加载与扩展。

设计器层基于 uni-app 的<movable-area>与<movable-view>实现拖拽定位功能。用户从组件库拖拽元素至画布时，前端通过@touchstart 与@touchend 事件监听坐标变化，动

态计算组件在画布中的位置与层级关系。画布状态通过 Pinia 存储，包含组件列表、选中组件 ID 及布局参数，通过计算触点坐标与画布容器的相对位置，动态生成组件实例。

用户在配置面板层选中画布中的组件后，右侧面板动态渲染对应属性的表单控件。通过 v-model 与组件属性双向绑定。配置面板采用动态表单渲染技术，根据组件类型加载对应的属性配置表单。

（2）交互逻辑实现

当用户释放拖拽的组件时，前端计算画布坐标系中的落点位置，并遍历现有组件树判断是否进入容器区域。若落点位于容器内，则递归调整父组件引用；若为自由区域，则创建新的布局容器。

配置面板的表单控件与画布中选中组件的属性通过响应式对象关联。数据模型通过解析表单构建模块中配置的字段信息，构建动态的响应式数据模型：系统遍历画布中所有表单组件提取 fieldName 字段标识，生成包含 formData.\${fieldName} 属性的 reactive({ formData: {} }) 基础对象；同时根据组件配置的校验规则，自动生成对应的 rules 校验规则对象并绑定至 uni-forms 组件的 :rules 属性，当用户修改配置值时，实时触发画布组件的重新渲染。

基于 JavaScript 模板字符串动态生成 Vue 模板代码。系统根据组件类型预定义模板片段，支持递归嵌套层级结构与条件编译逻辑。导出时，遍历组件树的 JSON 配置，将字段名、属性值注入模板字符串的占位符中，生成符合 uni-app 规范的 Vue 单文件组件，并自动注入数据绑定与校验规则。

最终生成的 Vue 文件包含完整的数据模型、校验规则以及表单模板代码，确保数据绑定与校验逻辑的完整集成，可直接运行于 uni-app 项目环境中。

为实现表单构建模块的可视化拖拽与动态配置功能，系统通过以下步骤实现组件管理、状态同步以及代码生成逻辑：

（1）组件元数据配置，用于定义基础表单组件的可配置属性和代码模板；示例代码如表 5-16 所示，通过 JSON 结构定义组件的可配置属性和代码模板，支持动态扩展新组件类型。

表 5-16 组件元数据配置示例代码

```
// 基础表单组件元数据定义
export const FORM_COMPONENTS = [
  {
    type: 'uni-input', // 组件类型标识
    label: '文本输入框', // 显示名称
    props: {           // 可配置属性及默认值
      fieldName: { type: 'string', required: true, default: 'field1' },
      label: { type: 'string', default: '请输入内容' },
      rules: { type: 'array', default: [{ required: true, message: '字段必填' }] },
    },
  },
]
```

表 5-16 组件元数据配置示例代码（续）

```
// 代码生成模板（支持占位符替换）
template: `
  <uni-form-item :label="props.label">
    <uni-input
      v-model="formData.{{fieldName}}"
      :rules="rules.{{fieldName}}"
    />
  </uni-form-item> `
},// 其他组件定义（如下拉框、日期选择器等）
];
```

（2）拖拽定位逻辑，基于 uni-app 的 movable-view 实现组件位置动态计算；示例代码如表 5-17 所示，利用 dataTransfer 传递组件类型标识，计算落点坐标后生成组件实例，确保位置精确性。

表 5-17 拖拽定位逻辑示例代码

```
// 处理组件拖拽释放事件
function handleComponentDrop(event) {
  const componentType = event.dataTransfer.getData('component-type');
  const meta = FORM_COMPONENTS.find(c => c.type === componentType);
  // 生成唯一组件实例
  const newComponent = {
    id: `comp_${Date.now()}`, // 唯一标识
    type: meta.type,
    position: { x: event.offsetX, y: event.offsetY }, // 落点坐标
    props: { ...meta.props } // 初始化属性
  };
  // 存储至状态管理
  formStore.addComponent(newComponent);
}
```

（3）代码生成算法，通过模板引擎将组件配置映射为可运行的 Vue 单文件代码。示例代码如表 5-18 所示。基于模板替换技术，将用户配置的字段名、校验规则注入预置模板，生成可直接运行的 Vue 文件。

表 5-18 表单代码生成算法示例代码

```
// 生成 Vue 单文件组件代码
export function generateFormCode(components) {
  let templateCode = '';
  let scriptCode = 'export default {\n  data() {\n    return {\nformData: {\n  let rulesCode = '      rules: {\n';
```

表 5-18 表单代码生成算法示例代码（续）

```
// 遍历组件树生成代码片段
components.forEach(comp => {
  const meta = FORM_COMPONENTS.find(c => c.type === comp.type);
  // 替换模板占位符
  let code = meta.template
    .replace(/{{fieldName}}/g, comp.props.fieldName)
    .replace(/{{label}}/g, comp.props.label);
  templateCode += code + '\n';
// 收集数据模型与校验规则
  scriptCode += '\n      ${comp.props.fieldName}: ",";';
  rulesCode += `      ${comp.props.fieldName}:
${JSON.stringify(comp.props.rules)},\n`;
}); // 组合完整代码
return `<template><uni-form>  ${templateCode}</uni-form></template>
<script>${scriptCode},${rulesCode}</script>`;}
```

用户主要操作包括拖拽组件到画布、配置已选中的画布组件参数、移除画布中的单个组件、清空画布区域内的所有组件、导出 uni-app 表单代码到 vue 文件、将代码复制到剪切板。表单构建的用户操作序列图如图 5-19 所示。

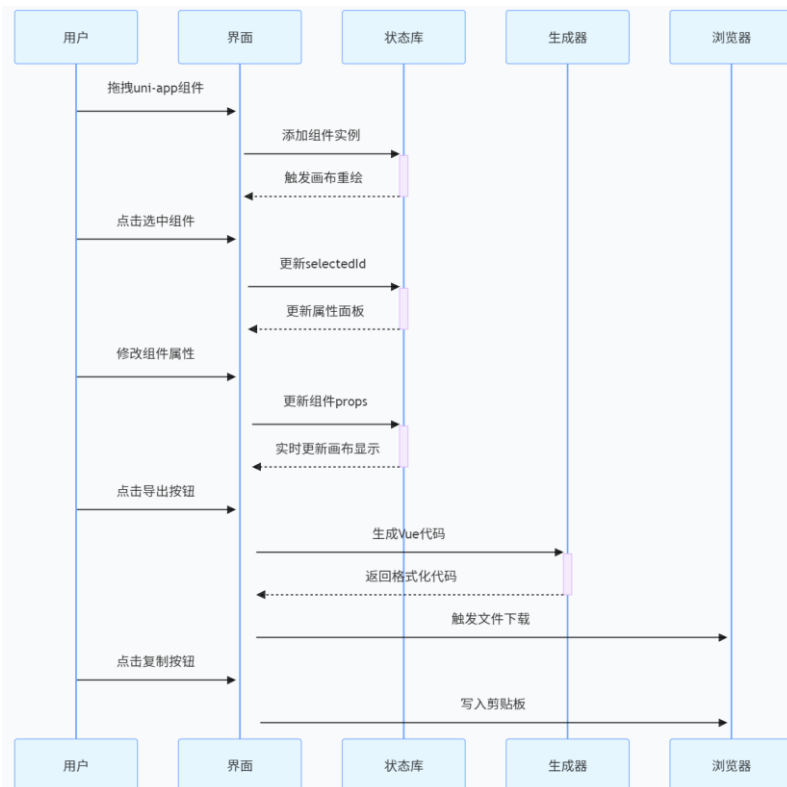


图 5-19 表单构建序列图

uni-app 表单构建界面如图 5-20 所示。其中左侧为组件区域，右侧为组件属性和表单属性配置区域，中间为画布区域。



图 5-20 表单构建界面

5.5.2 代码生成

代码生成模块主要是在 RuoYi 开源项目^[41]的代码生成模块基础之上，结合本文使用到的相关前后端技术做了针对性改进，基于 FreeMarker 模板引擎^[42]，实现了前后端代码的动态渲染与生成，核心流程包括元数据解析、模板绑定与文件输出。元数据解析主要是指从数据库表结构中提取表名、字段信息等相关数据生成结构化的 TableMeta 对象，同时基于字段注释中的特定标记，自动提取校验规则与绑定逻辑，通过占位符参数与模板文件绑定，映射为 Java 对象，供模板渲染使用。表和字段元数据模型的示例代码如表 5-19 所示。

表 5-19 字段映射示例代码

```
// TableMeta.java: 表元数据模型
public class TableMeta {
    private String tableName;        // 表名
    private String tableComment;    // 表注释
    private List<ColumnMeta> columns; // 字段列表 }

// ColumnMeta.java: 字段元数据模型
public class ColumnMeta {
    private String columnName;      // 字段名
    private String dataType;        // 数据库类型
    private String javaType;        // 映射的 Java 类型
    private String columnComment;  // 字段注释 }
```

字段映射的示例代码如表 5-20 所示。

表 5-20 字段映射示例代码

```
// TypeMapping.java: 数据库类型到 Java 类型的映射
public class TypeMapping {
    private static final Map<String, String> TYPE_MAP =
new HashMap<>();
    static {
        TYPE_MAP.put("varchar", "String");
        TYPE_MAP.put("int", "Integer");
        TYPE_MAP.put("bigint", "Long");
        TYPE_MAP.put("datetime", "LocalDateTime");
        // 其他类型映射...}
    public static String getJavaType(String dbType) {
        return TYPE_MAP.getOrDefault(dbType, "Object");
    }
}
```

系统中定义的模板文件包括控制类文件、服务层文件、实体类文件、uni-app 页面文件、前端请求脚本文件。通过预定义的占位符可为模板文件动态注入元数据。模板渲染时，根据用户配置的相关规则将元数据注入模板，生成目标代码并输出为文件。模板渲染示例代码如表 5-21 所示。

表 5-21 模板渲染示例代码

```
// CodeGenerator.java: 模板渲染与文件生成
public class CodeGenerator {
    public void generateCode(TableMeta tableMeta, String templatePath, String
outputPath) {
        Configuration cfg = new Configuration(Configuration.VERSION_2_3_32);
        cfg.setClassForTemplateLoading(this.getClass(), "/templates");
        try { // 加载模板文件
            Template template = cfg.getTemplate(templatePath);
            // 绑定数据模型
            Map<String, Object> data = new HashMap<>();
            data.put("packageName", "com.example");
            data.put("className", tableMeta.getTableName());
            data.put("tableName", tableMeta.getTableName());
            data.put("tableComment", tableMeta.getTableComment());
            data.put("columns", tableMeta.getColumns());
            // 渲染并输出文件
            Writer writer = new FileWriter(outputPath);
            template.process(data, writer);
            writer.close();
        } catch (Exception e) {
            throw new RuntimeException("代码生成失败: " + e.getMessage());
        }
    }
}
```

代码生成功能的用户操作序列图如图 5-21 表示。

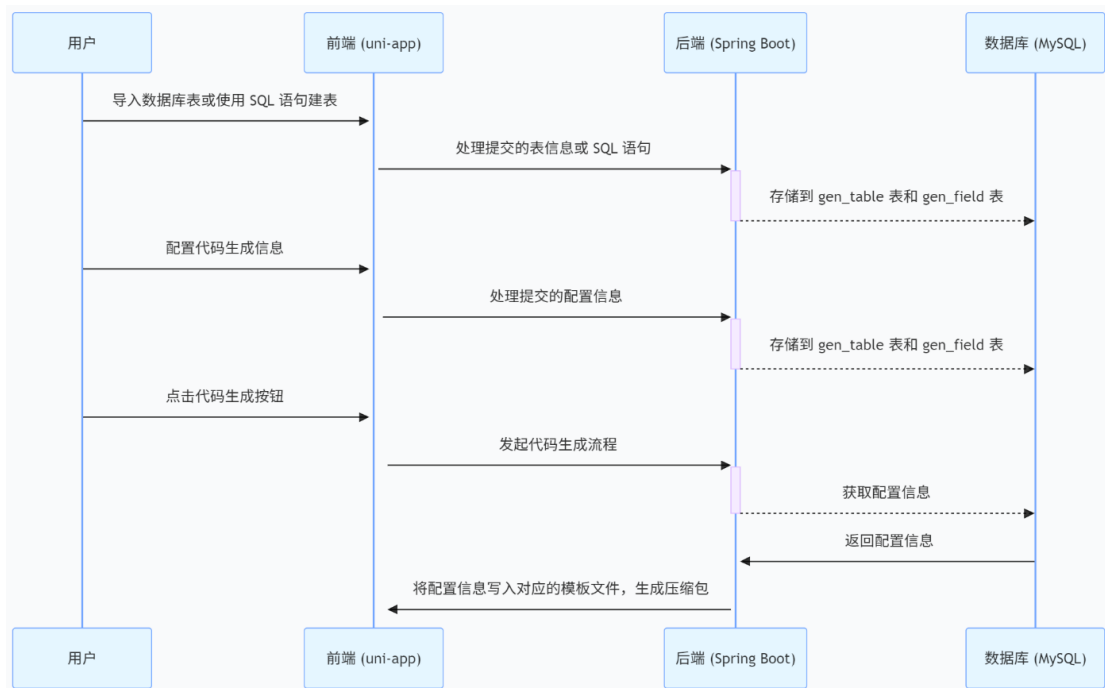


图 5-21 用户操作序列图

代码生成模块，涉及的主要 Java 类包括模板引擎类 `CodeGenerator`、文件工具类 `FileUtils`、源数据解析类 `MetaDataParser`、`CodeGenParams` 动态参数类、`GenTableController`。模板引擎类通过将动态数据通过模板变量绑定到模板文件，生成满足需求的目标代码。`GenTableController` 用于处理 HTTP 请求，主要接口如表 5-22 所示。

表 5-22 代码生成主要接口

接口	请求方式	请求参数	返回参数	描述
<code>/gen/tables</code>	GET	<code>TableQuery</code>	<code>Result<PageResult<TableVO>></code>	分页查询
<code>/gen/tables/import</code>	POST	<code>TableImportDTO</code>	<code>Result<Void></code>	导入表
<code>/gen/tables/sql</code>	POST	<code>SqlCreateTableDTO</code>	<code>Result<Void></code>	SQL 建表
<code>/gen/tables</code>	DELETE	<code>BatchDeleteDTO</code>	<code>Result<Void></code>	批量删除
<code>/gen/tables/{name}/config</code>	POST	<code>GenConfigTemplate</code>	<code>Result<GenConfigVO></code>	生成配置
<code>/gen/tables/{name}/code</code>	POST	<code>CodeGenParams</code>	<code>Result<FileDownloadVO></code>	生成代码

(1) 表结构解析与导入

用户可通过两种方式导入表结构：在数据库表列表中选择目标表，前端发送 `TableImportDTO` 对象至 `/gen/tables/import` 接口，后端通过 `MetaDataParser` 解析表结构，提取表名、字段、主键等信息，存储至 `gen_table` 与 `gen_field` 元数据表，并校验表是否已存在；或者输入 SQL 建表语句，前端提交 `SqlCreateTableDTO` 至 `/gen/tables/sql` 接口，

后端执行 DDL 语句创建数据库表后同步解析元数据。导入成功后，表信息将展示在代码生成列表页。

(2) 批量删除已导入的表

用户可批量选择已导入的表进行删除操作，前端提交 BatchDeleteDTO 至/gen/tables 接口，后端删除数据库表中代码生成配置表和字段表的相关记录。

(3) 代码生成配置

在代码生成列表页点击“生成配置”按钮，进入生成规则配置界面。前端调用/gen/tables/{name}/config 接口获取 GenConfigVO 初始配置信息。用户可对生成的目录结构、文件名称、文件内容的相关模板参数进行配置。配置完成后，系统将映射关系持久化至数据库表。代码字段配置界面如图 5-22 所示。

序号	字段名	字段类型	Java属性名	属性类型	插入	编辑	列表	查询	查询方式	必填	组件类型	字典类型
1	student_id	bigint	studentId	Long	☒	☐	☐	☐	=	☐	文本框	请选择
2	student_name	varchar(50)	studentN	String	☒	☒	☒	☒	LIKE	☒	文本框	请选择
3	gender	char(1)	gender	String	☒	☒	☒	☒	=	☐	文本框	请选择
4	age	int	age	Long	☒	☒	☒	☒	=	☐	文本框	请选择
5	phone	varchar(11)	phone	String	☒	☒	☒	☒	=	☐	文本框	请选择
6	email	varchar(50)	email	String	☒	☒	☒	☒	=	☐	文本框	请选择
7	status	char(1)	status	String	☒	☒	☒	☒	=	☐	单选框	请选择
8	create_bv	varchar(64)	createBv	String	☒	☐	☐	☐	=	☐	文本框	请选择

确定取消

图 5-22 字段配置界面

(4) 生成代码

用户配置生成规则后，点击“生成代码”按钮，触发/gen/tables/{name}/code 接口。后端通过 CodeGenParams 获取配置参数与表元数据；然后调用 FreeMarker 引擎将 TableMeta 对象绑定到模板文件；生成前端 uni-app 页面、前端接口文件、后端各层 Java 文件以及 xml 映射文件；最后使用 FileUtils 将代码打包为 ZIP，通过 FileDownloadVO 返回下载链接。代码生成界面如图 5-23 所示。

表名 创建时间 - 查询 重置

导入数据库表 + SQL建表 批量删除

☐	表名	表注释	实体类名	创建时间	更新时间	操作
☐	sys_student	学生信息表	SysStudent	2025-05-21 14:44:46	2025-05-22 09:58:12	生成配置 生成代码 删除

共 1 条 10条/页 < 1 > 前往 1 页

图 5-23 代码生成界面

5.6 本章小结

本章详细阐述了系统的开发环境配置与核心模块实现，首先介绍了系统的开发环境和项目搭建，然后介绍了主要模块的详细设计与实现。系统配置模块通过动态参数管理与字典维护实现了业务规则灵活调整；权限管理模块基于 RBAC 模型实现了动态的细粒度权限控制；系统监控模块集成 Druid、Oshi 实现运行状态实时监控与日志管理；开发工具模块提供了 uni-app 组件的可视化表单构建功能和基于 FreeMarker 的代码生成功能。

第六章 系统测试

6.1 测试环境

在进行系统测试之前，首先基于中小型企业实际开发场景构建测试环境，主要包括硬件环境和软件环境。

(1) 硬件环境：服务器选用 Intel Xeon E5-2678 处理器，选择 32GB 内存与 1TB SSD 存储，客户端选用 Intel i7-10700 处理器，网络环境配置为 1Gbps 局域网与 1Gbps 外网带宽，模拟企业内网开发与公网访问混合场景。测试环境的具体硬件配置如表 6-1 所示。

表 6-1 硬件配置表

分类	配置参数
服务端	CPU: Intel Xeon E5-2678、内存: 32GB DDR4、 存储: 1TB SSD
客户端	CPU: Intel i7-10700、内存: 16GB、操作系统: Windows 10
网络环境	内网带宽: 1Gbps、外网带宽: 1Gbps

(2) 软件环境：软件环境可分为运行环境和测试工具，运行环境主要包括前端 Node.js 环境、后端 JDK 环境、MySQL 数据库、Redis 缓存；测试工具包括用于模拟系统并发的 JMeter 和用于进行接口功能验证的 Postman。测试环境的具体软件配置如表 6-2 所示。

表 6-2 软件配置表

分类	工具/版本
前端环境	Node.js 16.20.2
后端环境	JDK 17.0.8
数据存储工具	MySQL 8.0.34、Redis 6.2.12
浏览器	Chrome 120、Firefox 115、IE 11
测试工具	JMeter 5.6.2、Postman 10.18.0、OWASP ZAP、 Wireshark

6.2 系统功能测试

本节对系统的主要模块进行功能测试。

(1) 字典类型管理功能测试用例如表 6-3 所示。

表 6-3 字典类型测试用例表

测试编号	1-1
测试目的	测试字典类型管理功能
前置条件	具有字典类型管理的访问权限
测试步骤	1.点击“新增”按钮，填写字典类型相关信息，点击“确定”按钮； 2. 点击“修改”按钮，修改字典类型相关信息，点击“确定”按钮； 3. 点击“删除”按钮，在弹窗中点击“确定”按钮； 4.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 5.填写条件查询表单的相关信息，点击“查询”按钮。
预期结果	1.字典类型表中增加对应记录； 2.字典类型表中修改对应记录； 3.字典类型表中删除对应记录； 4.字典类型表中删除对应的多条记录； 5.界面上显示对应的查询结果。
测试结果	测试通过。

(2) 字典内容管理功能测试用例如表 6-4 所示。

表 6-4 字典内容测试用例表

测试编号	1-2
测试目的	测试字典内容管理功能
前置条件	具有字典内容管理的访问权限
测试步骤	1.点击“新增”按钮，填写字典内容相关信息，点击“确定”按钮； 2. 点击“修改”按钮，修改字典内容相关信息，点击“确定”按钮； 3. 点击“删除”按钮，在弹窗中点击“确定”按钮； 4.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 5.填写条件查询表单的相关信息，点击“查询”按钮。
预期结果	1.字典内容表中增加对应记录； 2.字典内容表中修改对应记录； 3.字典内容表中删除对应记录； 4.字典内容表中删除对应的多条记录； 5.界面上显示对应的查询结果。
测试结果	测试通过。

(2) 参数管理功能测试用例如表 6-5 所示

表 6-5 参数管理测试用例表

测试编号	1-3
测试目的	测试参数管理功能
前置条件	具有参数管理的访问权限
测试步骤	1.点击“新增”按钮，填写系统参数相关信息，点击“确定”按钮； 2. 点击“修改”按钮，修改系统参数相关信息，点击“确定”按钮； 3. 点击“删除”按钮，在弹窗中点击“确定”按钮； 4.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 5.填写条件查询表单的相关信息，点击“查询”按钮。
预期结果	1.系统参数表中增加对应记录； 2.系统参数表中修改对应记录； 3.系统参数表中删除对应记录； 4.系统参数表中删除对应的多条记录； 5.界面上显示对应的查询结果。
测试结果	测试通过。

(4) 用户管理功能测试用例如表 6-6 所示。

表 6-6 用户管理测试用例表

测试编号	1-4
测试目的	测试用户管理功能
前置条件	具有用户管理的访问权限
测试步骤	1.点击“新增”按钮，填写用户相关信息，点击“确定”按钮； 2.点击“修改”按钮，修改用户相关信息，点击“确定”按钮； 3.点击“删除”按钮，在弹窗中点击“确定”按钮； 4.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 5.填写条件查询表单的相关信息，点击“查询”按钮； 6.点击“更多”按钮，点击“分配角色”按钮，勾选表格中的角色，点击确定。
预期结果	1.用户表中增加对应记录； 2.用户表中修改对应记录； 3.用户表中删除对应记录； 4.用户表中删除对应的多条记录； 5.界面上显示对应的查询结果； 6.用户角色表增加对应的关联记录。
测试结果	测试通过。

(5) 角色管理功能测试用例如表 6-7 所示。

表 6-7 角色管理测试用例表

测试编号	1-5
测试目的	测试角色管理功能
前置条件	具有角色管理的访问权限
测试步骤	1.点击“新增”按钮，填写角色相关信息，点击“确定”按钮； 2.点击“修改”按钮，修改角色相关信息，点击“确定”按钮； 3.点击“删除”按钮，在弹窗中点击“确定”按钮； 4.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 5.填写条件查询表单的相关信息，点击“查询”按钮； 6.点击“更多操作”按钮，点击“分配用户”按钮，点击“取消关联”按钮，点击“添加用户”按钮，勾选表格中的用户，点击“确定”按钮； 7.点击“修改”按钮，修改角色关联的菜单树，点击“确定”按钮
预期结果	1.角色表中增加对应记录； 2.角色表中修改对应记录； 3.角色表中删除对应记录； 4.角色表中删除对应的多条记录； 5.界面上显示对应的查询结果； 6.用户角色表变更对应的关联记录； 7.角色菜单表变更对应的关联记录。
测试结果	测试通过。

(6) 菜单管理模块功能测试用例如表 6-8 所示

表 6-8 菜单管理测试用例表

测试编号	1-6
测试目的	测试菜单管理功能
前置条件	具有菜单管理的访问权限
测试步骤	1.点击“新增”按钮，填写菜单相关信息，点击“确定”按钮； 2.点击“修改”按钮，修改菜单相关信息，点击“确定”按钮； 3.点击“删除”按钮，在弹窗中点击“确定”按钮； 4.填写条件查询表单的相关信息，点击“查询”按钮。
预期结果	1.菜单表中增加对应记录； 2.菜单表中修改对应记录； 3.菜单表中删除对应记录； 4.界面上显示对应的查询结果。
测试结果	测试通过。

(7) 数据监控功能测试用例如表 6-9 所示。

表 6-9 数据监控测试用例表

测试编号	1-7
测试目的	测试数据监控功能
前置条件	具有数据监控的访问权限
测试步骤	点击数据监控菜单。
预期结果	正确显示数据监控页面和相关数据。
测试结果	测试通过。

(8) 性能监控模块功能测试用例如表 6-10 所示。

表 6-10 性能监控测试用例表

测试编号	1-8
测试目的	测试性能监控功能
前置条件	具有性能监控的访问权限
测试步骤	点击性能监控菜单。
预期结果	正确显示性能监控页面和相关数据。
测试结果	测试通过。

(9) 登录日志模块功能测试用例如表 6-11 所示。

表 6-11 登录日志测试用例表

测试编号	1-9
测试目的	测试登录日志管理功能
前置条件	具有登录日志的访问权限
测试步骤	1.点击“清空日志”按钮； 2.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 3.填写条件查询表单的相关信息，点击“搜索”按钮。 4.退出后重新登录
预期结果	1.登录日志表中清空所有登录日志记录； 2.登录日志表中删除对应的多条记录； 3.界面上显示对应的查询结果； 4.界面上显示本次登录的日志记录。
测试结果	测试通过。

(10) 操作日志管理模块测试用例如表 6-12 所示。

表 6-12 操作日志测试用例表

测试编号	1-10
测试目的	测试操作日志管理功能
前置条件	具有操作日志的访问权限
测试步骤	1.点击“清空日志”按钮； 2.勾选表格中多行数据，点击“批量删除”按钮，在弹窗中点击“确定”按钮； 3.填写条件查询表单的相关信息，点击“搜索”按钮； 4.点击“详细信息”按钮。
预期结果	1.登录日志表中清空所有登录日志记录； 2.登录日志表中删除对应的多条记录； 3.界面上显示对应的查询结果； 4.弹窗显示对应的详细信息。
测试结果	测试通过。

(11) 表单构建功能模块测试用例如表 6-13 所示。

表 6-13 表单构建测试用例表

测试编号	1-11
测试目的	测试表单构建功能
前置条件	具有表单构建模块的访问权限
测试步骤	1.从左侧组件列表中拖拽一个组件到中间画布区域的目标位置，释放鼠标左键； 2.选中画布中的每个组件，在右侧面板配置相关属性； 3.点击“导出到 vue 文件”按钮，在弹窗中设置文件名和表单类型，点击“确定”按钮 4.点击“复制到剪切板”按钮； 5.点击组件的“删除组件”按钮； 6.点击“清空组件”按钮。
预期结果	1.画布中显示新添加的组件； 2.组件属性值同步更改； 3.表单成功导出至文件； 4.表单代码成功复制到剪切板； 5.组件从画布中被移除； 6.画布中的组件被清空。
测试结果	测试通过。

(12) 代码生成功能测试用例如表 6-14 所示。

表 6-14 代码生成测试用例表

测试编号	1-12
测试目的	测试代码生成功能
前置条件	具有代码生成模块的访问权限
测试步骤	1.点击“导入数据库表”按钮，勾选数据库表，点击“确定”按钮； 2.点击“SQL 建表”按钮，输入相应 SQL 语句，点击“确定”按钮； 3.填写条件查询表单的相关信息，点击“查询”按钮； 4.点击“删除”按钮； 5.点击“批量删除”按钮； 6.点击“生成配置”按钮； 7.点击“生成代码”按钮。
预期结果	1.代码生成配置表和代码生成字段表中增加对应记录； 2.代码生成配置表和代码生成字段表中增加对应记录； 3.界面上显示对应的查询结果； 4.代码生成配置表和代码生成字段表中删除对应记录； 5.代码生成配置表和代码生成字段表中移除对应记录； 6.显示生成配置页面； 7.生成代码到文件。
测试结果	测试通过。

6.3 系统非功能测试

本节对系统的安全性、兼容性和性能进行测试。

(1) 安全性测试

安全性测试是为了验证系统在身份认证、数据防护等维度上的可靠性。测试覆盖权限拦截、风险防御以及加密传输等核心场景，验证系统安全机制的有效性。具体测试内容及结果如表 6-15 所示。

表 6-15 安全性测试用例表

测试项	测试方法	测试工具	预期结果	实测结果	通过率
权限拦截测试	模拟用户访问未授权的接口	Postman	返回 403 错误	拦截成功率 100%	100%
风险防御测试	注入 SQL 语句及恶意脚本	OWASP ZAP	请求被拦截	攻击阻断率 100%	100%
加密传输测试	抓包分析登录请求	Wireshark	无明文泄露	TLS 1.3 已启用	100%
密码存储安全	查询用户表的密码字段	Navicat	BCrypt 动态盐值哈希格式	符合加密规范	100%

(2) 兼容性测试

兼容性测试主要是为了验证系统在不同环境下显示和操作的一致性，在本系统中，主要是指对同一设备不同浏览器下的布局适配测试、UI 组件测试以及对老旧浏览器支持度的测试。具体测试内容及结果如表 6-16 所示。

表 6-16 兼容性测试用例表

测试维度	测试环境	测试方法	预期指标	实测结果	通过率
跨浏览器布局适配	Chrome 120/Firefox 115/IE 11、Chrome、Safari	响应式设计模式测量组件偏移	最大误差 ≤5%	平均误差 2.4%	100%
老旧浏览器支持	IE 11	Babel ES5 转译与功能验证	核心功能正常	表单提交成功率 100%	100%

(3) 性能测试

性能测试主要是为了验证系统在高并发与复杂业务场景下的稳定性、响应效率及服务承载能力。测试采用 JMeter 工具模拟实际用户行为，从并发用户数、平均响应时间、数据库负载以及测试场景的通过率来验证系统是否满足需求分析中定义的性能指标。具体测试内容及结果如表 6-17 所示。

表 6-17 性能测试用例表

测试场景	并发用户数	平均响应时间(ms)	数据库负载(%)	通过率
用户登录	100	120	20	99.8%
字典数据查询	200	50	20	99.5%
角色权限分配	50	150	35	98.5%
表单构建页面渲染	150	90	10	100%
代码生成接口调用	80	180	30	97.2%
系统监控接口调用	300	100	25	99.0%

6.4 本章小结

本章对系统进行了功能测试和非功能测试，首先介绍了系统的测试环境，然后对系统的主要功能进行了测试并给出了对应的测试用例。针对系统的非功能需求，从安全性、兼容性、性能三个方面进行了测试。系统测试结果表明，系统基本满足了预期的目标需求，可用于后续的开发实践。

第七章 总结与展望

本文针对中小型企业 Web 应用开发中普遍面临的基础功能重复开发、跨平台适配成本高、开发周期紧张等关键问题，设计并实现了一个轻量级 Web 应用开发平台。主要工作内容如下：

(1) 经过深入的调研与分析，阐述了基于 uni-app 与 Spring Boot 框架的 Web 应用开发平台的研究背景及意义，分析了跨平台技术和 Web 应用开发平台的发展现状，明确了论文的主要研究内容和组织结构；

(2) 介绍了论文涉及到的相关技术，对 uni-app 框架、Pinia 状态管理库、Spring Boot 框架、Sa-Token 框架和 FreeMarker 模板引擎、MyBatis-Plus 框架进行了概述；

(3) 根据 Web 应用开发现状和实际应用场景，分析了系统的功能性需求和非功能性需求；

(4) 在系统的总体设计中，将系统整体架构划分为客户端层、前端层、后端层、数据存储层，将系统划分为系统配置模块、权限管理模块、系统监控模块、开发工具模块，根据业务需求，基于 ER 模型完成了数据库设计；

(5) 基于前后端分离的开发模式，对系统的各个功能模块进行了详细设计与实现，主要功能包括字典管理、参数管理、用户管理、角色管理、菜单管理、日志管理、表单构建和代码生成；

(6) 对实现的系统进行了多维度的测试，并将系统运用于实际项目的开发实践，验证了系统预期目标和实际应用价值。

本文设计并实现的基于 uni-app 与 Spring Boot 框架的 Web 应用开发平台，在一定程度上解决了基础功能重复开发、技术适配性不足、功能冗余等问题，提高了应用开发效率，但系统仍有很多需要完善和改进的方面，主要包括以下几点：

(1) 系统提供的功能不够丰富，仅提供了一些基本的通用功能，在应对复杂业务场景时存在局限，后续将集成工作流引擎和任务调度等模块，增强平台通用性，对企业的多样化需求提供有效的支持；

(2) 当前平台的表单构建和代码生成功能采用的是基于模板的方式。后续将探索融合低代码可视化与 AI 大模型技术，支持通过更灵活的需求描述进行智能化代码生成，进一步提升复杂业务场景的开发效率；

(3) 系统采用单体架构，契合中小型项目的轻量化需求，但在用户量激增等高并发场景下的水平扩展能力不足。后续应根据实际情况，逐步拆分关键服务为微服务，降低迁移风险，提升高并发场景下的弹性伸缩能力；

(4) 系统仅对前端通用的 uni-app 组件和 API 进行了封装，未针对各平台专有 API 进行有效处理，后续将支持多端原生 API 封装，降低开发者适配成本。

参考文献

- [1] 苏钰玲. 一种轻量级安全 Web 开发框架的研究与实现[D]. 华北电力大学(北京), 2023.
- [2] 谭少华. Java Web 管理信息系统快速开发平台的设计与实现[D]. 电子科技大学, 2018.
- [3] 庄海. 基于自适应布局的跨平台融媒体运营管理系统的前端设计与实现[J]. 现代电视技术, 2024,(05):71-76.
- [4] Juan Li,Zhicheng Dou,Yutao Zhu, Xiaochen Zuo, Ji-Rong Wen. Deep cross-platform product matching in e-commerce[J]. Information Retrieval Journal, 2020, 23(11).
- [5] 王硕,祁欣颖. React Native 的跨平台移动应用开发实践[J]. 信息与电脑(理论版), 2024, 36(07):119-121.
- [6] 邓皓瀚. 基于 Flutter 的跨平台移动 APP 开发前景研究[J]. 信息与电脑(理论版), 2019(15):197-199.
- [7] 向治洪. React Native 移动开发实战[M]. 人民邮电出版社:202005.464.
- [8] 薄彦彦,王秀慧,赵倩倩. 基于 Flutter 的“文旅大同”APP 设计与开发[J]. 山西大同大学学报(自然科学版), 2023,39(04):45-48.
- [9] 彭婕. 应用 uni-app 框架的综合服务 APP 的研究与设计[J]. 信息与电脑(理论版), 2024,36(09):143-145.
- [10] 伍展宏,陈展豪,李商好. 基于 Jeecg Boot+Vue3 的在线证书考试系统设计与实现[J]. 网络安全和信息化, 2024, (02):95-97.
- [11] Martins R,Caldeira F, Sa F, et al. An overview on how to develop a low-code application using OutSystems[C]//2020 International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE). 2020.
- [12] Behi Z,Ng K T W, Richter A, et al. Exploring the untapped potential of solar photovoltaic energy at a smart campus: Shadow and cloud analyses:[J]. SAGE PublicationsSage UK: London, England, 2021, 22(4): 12.
- [13] 张巍. 基于钉钉宜搭的中小企业低代码平台的应用研究[D]. 北京邮电大学, 2022.
- [14] Razzaq M A,Mahar J A,Ahmad M, et al. The 3-Axis Scalable Service-Cloud Resource Modeling for Burst Prediction Under Smart Campus Scenario[J]. IEEE Access, 2021, 9(2): 116927-116941.
- [15] 苏伟,国建勋,冯宽. 低代码开发平台发展现状及标准化研究[J]. 信息技术与标准化, 2024(Z1):17-21.
- [16] Ullrich C,Lata T, Geyer-Klingenberg J. Celonis Studio—A Low-Code Development Platform for Citizen Developers[J]. 2021.
- [17] Denko Blaž,Pečnik Špela,Fister Jr.Iztok. A Comprehensive Comparison of Hybrid Mobile Application Development Frameworks[J]. International Journal of Security and Privacy in Pervasive Computing (IJSPPC),2021.
- [18] 吴昌政. 基于前后端分离技术的 web 开发框架设计[D]. 南京邮电大学, 2020.

- [19] 刘启伟. 基于 Vue.js 框架的 Web 前端开发工具的设计与实现[D]. 北京邮电大学, 2021.
- [20] 郭文学. Web 应用快速开发工具设计与实现[D]. 山东大学, 2022.
- [21] 康雯璐. 可视化低代码开发平台的研究与实现[D]. 北京邮电大学, 2023.
- [22] 王雨霏. 数据可视化低代码 SaaS 服务平台的设计与实现[D]. 北京邮电大学, 2024.
- [23] 付兴. 基于 uni-app 的污水处理厂掌上系统的设计与开发[D]. 湖州师范学院, 2023.
- [24] 褚润润. 基于 Uni-App 的跨平台企业办事通的设计与实现[D]. 华中科技大学, 2022.
- [25] DCloud. uni-app 官方文档[EB/OL]. [2024-03-15]. <https://uniapp.dcloud.net.cn/>.
- [26] 邹聪,张浩洋. 基于 Vue3 的数据申请管理系统设计与实现[J]. 现代计算机, 2024,30(23):211-214.
- [27] GOMES F, SOARES P, ARAÚJO A A. Examining the performance implications of design patterns in front-end web development: A preliminar comparative study with react and vue.js[C]//Proceedings of the 29th Brazilian Symposium on Multimedia and the Web. New York, NY, USA: Association for Computing Machinery, 2023: 255–259.
- [28] Mythily M, Raj A S A, Joseph I T. An analysis of the significance of spring boot in the market [C]//2022 International Conference on Inventive Computation Technologies (ICICT). IEEE, 2022: 1277-1281.
- [29] Teng F,Wu Q. Design and implementation of the information system of retired veteran cadres bureau based on springboot framework[C]//2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE). IEEE, 2021: 87-92.
- [30] 包添文. 基于 Spring Boot 框架的社区管理系统设计与实现[D]. 中国地质大学(北京), 2023.
- [31] Sa-Token. Sa-Token 官方文档 [EB/OL]. [2024-03-17]. <https://sa-token.cc/>.
- [32] Matthew Tyson. How to secure REST with Spring Security[J]. InfoWorld.com, 2021.
- [33] 许梦微,王希胤. 基于 Shiro 的加密机制研究[J]. 电脑知识与技术, 2023, 19(22):95-97.
- [34] 滕杨凡. 互联网医院项目代码自动生成的设计与实现[D]. 北京邮电大学, 2021.
- [35] 陈海庆. 基于 BS 模式的产品数据管理系统的设计与实现[D]. 合肥工业大学, 2023.
- [36] 张斌,赵婷婷,张碧霞,等. 基于 RBAC 模型的中文医疗命名实体识别[J]. 天津科技大学学报, 2024, 39(05):56-62.
- [37] 王平,董国卿,夏凌云,等. 基于微服务架构的高校办事服务平台优化[J]. 计算技术与自动化, 2024, 43(04):135-140.
- [38] 曹啸鹏. 面向非功能需求的软件规模度量方法研究[J]. 信息技术与标准化, 2024, (11):38-44.
- [39] 胡荣,羊雪玲. 基于 Spring Boot 前后端分离 Web 系统的设计与实现[J]. 新能源与智能网联, 2024, (01):88-97.
- [40] 李瑞瑞. 长波数据库系统的设计与实现[D]. 西安电子科技大学, 2024.
- [41] Ruoyi. Ruoyi 官方文档[EB/OL]. [2024-06-30]. <https://doc.ruoyi.vip/>.
- [42] 梁嘉怡,卢雅敏,赵峻. 基于 FreeMarker 模板引擎的文档自动生成软件设计[J]. 信息与电脑(理论版), 2022,34(13):100-102.